

SISTEMAS OPERACIONAIS - ATIVIDADE 1 (AV1)

Processos, Threads, Escalonamento e Inferência Local com Ollama

Discente

Nome: Mariana de Jesus Silva

Curso: Engenharia da Computação

Instituição: Universidade Federal de Sergipe — UFS

Período: 2026.2

Modalidade: Individual

Trilha selecionada

Trilha A — Chat local: Ollama + Open WebUI

Modelo selecionado

Modelo: Llama3.2:3B

Família: Llama 3.2

Checkpoint/variante: Llama 3.2 3B — versão instruction-tuned

Número de parâmetros: 3,21 bilhões

Formato: GGUF

Quantização: Q4_K_M

Licença: Llama 3.2 Community License Agreement

Execução: Ollama

Hugging Face: <https://huggingface.co/meta-llama/Llama-3.2-3B-Instruct>

Repositório

GitHub: https://github.com/mariana-js/SO_UFS_2026_2_Silva_Mariana

Vídeo da atividade

URL: https://youtu.be/SH9_ovTPqbU

1. INTRODUÇÃO

Aplicações de inteligência artificial generativa executadas localmente dependem de diversos recursos e mecanismos do sistema operacional, incluindo processos, threads, memória, armazenamento, comunicação entre componentes, chamadas de sistema e escalonamento de CPU.

Esta atividade apresenta uma análise experimental de uma aplicação local de inteligência artificial composta pelo runtime Ollama e pela camada de aplicação Open WebUI. O ambiente utilizado foi o WSL 2, com Ubuntu 26.04.1 LTS.

O objetivo é observar como a execução local de um modelo de linguagem afeta os recursos computacionais disponíveis e relaciona essas observações aos conceitos estudados na disciplina de Sistemas Operacionais.

2. PROBLEMA E PERGUNTA NORTEADORA

A atividade busca investigar como a camada de aplicação, o modelo de linguagem, a quantidade de parâmetros, a quantização e a configuração de execução influenciam os processos, threads, uso de CPU, memória, armazenamento e responsividade de uma aplicação local de inteligência artificial generativa.

Pergunta norteadora

Como a camada de aplicação, o modelo de linguagem, a quantidade de parâmetros, a quantização e a configuração de execução afetam processos, threads, uso de CPU, memória, armazenamento e responsividade de um sistema local de IA generativa?

3. OBJETIVOS

3.1. Objetivo geral

Analisar o comportamento de uma aplicação local de inteligência artificial generativa sob a perspectiva de Sistemas Operacionais, observando processos, threads, chamadas de sistema e utilização de recursos computacionais.

3.2. Objetivos específicos

- Instalar e executar o Ollama em ambiente Linux utilizando WSL2.
- Instalar e executar o Open WebUI.
- Executar localmente um modelo de linguagem com até 10 bilhões de parâmetros.
- Identificar processos e threads relacionados à aplicação.
- Observar o consumo de CPU e memória.
- Investigar comunicação entre Open WebUI e Ollama.
- Observar chamadas de sistema relevantes durante a execução.
- Comparar diferentes configurações de execução.

- Realizar pelo menos 12 execuções mensuráveis.
 - Avaliar o impacto de diferentes cargas sobre os recursos do sistema.
 - Relacionar os resultados observados aos conceitos de Sistemas Operacionais.
 - Avaliar criticamente informações obtidas por ferramentas de inteligência artificial.
-

4. TRILHA DE APLICAÇÃO

4.1. Trilha escolhida

Trilha A — Chat local: Ollama + Open WebUI

4.2. Aplicação

Nome: Open WebUI

Repositório: <https://github.com/open-webui/open-webui>

Autor/organização: Open WebUI Community

Licença: MIT License

Versão, release ou commit utilizado: v0.11.4

Data de acesso: 22/09/2026

4.3. Finalidade

O Open WebUI foi utilizado como camada de aplicação para fornecer uma interface web para interação com o modelo de linguagem executado pelo Ollama.

A comunicação entre os componentes ocorre localmente, permitindo observar a interação entre uma aplicação web, o runtime responsável pela execução do modelo e os recursos disponibilizados pelo sistema operacional.

5. ARQUITETURA DA APLICAÇÃO

A arquitetura utilizada pode ser representada da seguinte forma:

Usuário -> Navegador Web -> Open WebUI -> HTTP / API local -> Ollama -> Modelo de linguagem (Llama 3.2 3B) -> CPU / Memória / Armazenamento.

O usuário interage com o modelo por meio da interface web disponibilizada pelo Open WebUI. A aplicação encaminha as solicitações ao Ollama por meio de comunicação HTTP local. O Ollama gerencia a execução do modelo Llama 3.2 3B e utiliza os recursos computacionais disponíveis no ambiente WSL2, principalmente CPU, memória RAM e armazenamento.

Essa arquitetura permite observar, de forma integrada, os componentes de uma aplicação de inferência local e os recursos do sistema operacional envolvidos durante sua execução. Além da interação pela interface web, a API local do Ollama possibilita a

realização de requisições diretamente ao runtime, o que será utilizado posteriormente para coleta de métricas e execução dos experimentos.

5.1. Componentes

Componente	Função
Navegador Web	Interface utilizada pelo usuário para acessar e interagir com o Open WebUI.
Open WebUI	Aplicação responsável pela interface web de chat e pelo encaminhamento das solicitações ao serviço de inferência.
Ollama	Runtime responsável por gerenciar e executar localmente o modelo de linguagem e disponibilizar sua API HTTP.
Modelo de linguagem	Llama 3.2 3B, responsável pelo processamento dos prompts e geração das respostas.
WSL2	Camada que disponibiliza um ambiente Linux integrado ao Windows para execução da aplicação.
Ubuntu	Distribuição Linux utilizada como sistema operacional do ambiente experimental.
CPU	Recurso computacional utilizado no processamento da aplicação e da inferência do modelo.
RAM	Memória utilizada temporariamente pelos processos, dados e componentes necessários à execução do modelo e das aplicações.
Armazenamento	Recurso utilizado para armazenar o modelo, arquivos da aplicação, dependências, configurações e demais dados persistentes.

6. AMBIENTE EXPERIMENTAL

6.1. Sistema operacional

Ambiente: WSL2

Distribuição: Ubuntu 26.04.1 LTS

Kernel: Linux Lenovo 6.18.33.2-microsoft-standard-WSL2

Comando utilizado:

uname -a

```
mariana@Lenovo:~$ uname -a
Linux Lenovo 6.18.33.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun 18 21:54:43 UTC 2026 x86_64 GNU/Linux
```

cat /etc/os-release

```
mariana@Lenovo:~$ cat /etc/os-release
PRETTY_NAME="Ubuntu 26.04.1 LTS"
NAME="Ubuntu"
VERSION_ID="26.04"
VERSION="26.04.1 LTS (Resolute Raccoon)"
VERSION_CODENAME=resolute
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=resolute
LOGO=ubuntu-logo
```

6.2. CPU

Modelo: Intel(R) Core(TM) i3-10110U CPU @ 2.10GHz

Núcleos físicos: 2

Threads: 4

Comandos utilizados:

lscpu

```
mariana@Lenovo:~$ lscpu
Architecture:                x86_64
CPU op-mode(s):              32-bit, 64-bit
Address sizes:                39 bits physical, 48 bits virtual
Byte Order:                   Little Endian
CPU(s):                       4
  On-line CPU(s) list:        0-3
Vendor ID:                    GenuineIntel
Model name:                   Intel(R) Core(TM) i3-10110U CPU @ 2.10GHz
CPU family:                   6
Model:                        142
Thread(s) per core:           2
Core(s) per socket:           2
Socket(s):                    1
Stepping:                     12
BogoMIPS:                     5184.01
Flags:                        fpu vme de pse tsc msr pae mce cx8 apic sep
                               mtrr pge mca cmov pat pse36 clflush mmx fx
                               sr sse sse2 ss ht syscall nx pdpe1gb rdtscp
                               lm arch_perfmon rep_good nopl xtopology cp
                               uid tsc_known_freq pni pclmulqdq vmx ssse3
                               fma cx16 pdcm pcid sse4_1 sse4_2 movbe popc
                               nt aes xsave avx f16c rdrand hypervisor lah
                               f_lm abm 3dnowprefetch ssbd ibrs ibpb stibp
                               ibrs_enhanced tpr_shadow ept vpid ept_ad f
                               sgsbase bml avx2 smep bmi2 erms invpcid rd
                               seed adx smap clflushopt xsaveopt xsavec xg
                               etbv1 xsaves vnm i md_clear flush_lld arch_c
                               apabilities
Virtualization features:
  Virtualization:              VT-x
  Hypervisor vendor:           Microsoft
  Virtualization type:         full
Caches (sum of all):
```

```

Caches (sum of all):
  L1d: 64 KiB (2 instances)
  L1i: 64 KiB (2 instances)
  L2: 512 KiB (2 instances)
  L3: 4 MiB (1 instance)
NUMA:
  NUMA node(s): 1
  NUMA node0 CPU(s): 0-3
Vulnerabilities:
  Gather data sampling: Unknown: Dependent on hypervisor status
  Ghostwrite: Not affected
  Indirect target selection: Mitigation; Aligned branch/return thunks
  Itlb multihit: KVM: Mitigation: Split huge pages
  L1tf: Not affected
  Mds: Not affected
  Meltdown: Not affected
  Mmio stale data: Mitigation; Clear CPU buffers; SMT Host state unknown
  Old microcode: Not affected
  Reg file data sampling: Not affected
  Retbleed: Mitigation; Enhanced IBRS
  Spec rstack overflow: Not affected
  Spec store bypass: Mitigation; Speculative Store Bypass disabled via prctl
  Spectre v1: Mitigation; usercopy/swapgs barriers and __user pointer sanitization
  Spectre v2: Mitigation; Enhanced / Automatic IBRS; IBPB conditional; PBR SB-eIBRS SW sequence; BHI SW loop, KVM SW loop
  Srbds: Unknown: Dependent on hypervisor status
  Tsa: Not affected
  Tsx async abort: Not affected
  Vmscape: Not affected

```

nproc

```

mariana@Lenovo:~$ nproc
4

```

6.3. Memória

RAM disponível ao ambiente: aproximadamente 5,7 GiB

Swap: 2,0 GiB

Comando:

free -h

```

mariana@Lenovo:~$ free -h
              total        used        free      shared  buff/cache   available
Mem:          5.7Gi        478Mi        5.2Gi        3.4Mi        217Mi
Swap:         2.0Gi           0B         2.0Gi

```

6.4. Armazenamento

Sistema de arquivos principal do WSL (/): aproximadamente 951 GB disponíveis no momento da coleta, com 1% de utilização.

Partição do Windows montada em /mnt/c: aproximadamente 16 GB disponíveis, com 94% de utilização.

Partição D: montada em /mnt/d: aproximadamente 214 GB disponíveis, com 5% de utilização.

Comandos:

`df -h`

```
mariana@Lenovo:~$ df -h
Filesystem      Size  Used Avail Use% Mounted on
none            2.9G   0 2.9G   0% /usr/lib/modules/6.18.33.2-microsoft-standard-WSL2
none            2.9G  4.0K 2.9G   1% /mnt/wsl
drivers          238G  222G   16G  94% /usr/lib/wsl/drivers
/dev/sdd        1007G   5.5G  951G   1% /
none            2.9G   36K 2.9G   1% /mnt/wslg
none            2.9G   0 2.9G   0% /usr/lib/wsl/lib
rootfs          2.9G   2.8M 2.9G   1% /init
none            2.9G  488K 2.9G   1% /run
none            2.9G   0 2.9G   0% /run/lock
none            2.9G   0 2.9G   0% /run/shm
none            2.9G   80K 2.9G   1% /mnt/wslg/versions.txt
none            2.9G   80K 2.9G   1% /mnt/wslg/doc
C:\             238G  222G   16G  94% /mnt/c
D:\             224G   9.7G  214G   5% /mnt/d
tmpfs           2.9G   0 2.9G   0% /tmp
none            1.0M   0 1.0M   0% /run/credentials/systemd-journald.service
none            1.0M   0 1.0M   0% /run/credentials/systemd-resolved.service
tmpfs           585M  12K  585M   1% /run/user/1000
none            1.0M   0 1.0M   0% /run/credentials/getty@tty1.service
```

`lsblk`

```
mariana@Lenovo:~$ lsblk
NAME MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda   8:0    0 356.9M  1 disk
sdb   8:16   0 159.4M  1 disk
sdc   8:32   0    2G   0 disk [SWAP]
sdd   8:48   0    1T   0 disk /mnt/wslg/distro
      /
```

6.5. GPU / VRAM

GPU: não foi identificada uma GPU NVIDIA disponível para uso pelo ambiente WSL2.

Dispositivo gráfico identificado pelo WSL: Microsoft Basic Render Driver.

VRAM dedicada disponível ao Ollama: não identificada.

Comandos:

`lspci | grep -Ei "vga|3d|display"`

```
mariana@Lenovo:~$ lspci | grep -Ei "vga|3d|display"
9bbc:00:00.0 3D controller: Microsoft Corporation Basic Render Driver
```

nvidia-smi

```
mariana@Lenovo:~$ nvidia-smi
Command 'nvidia-smi' not found, but can be installed with:
sudo apt install nvidia-utils-580 # version 580.159.03-0ubuntu0.
26.04.1, or
sudo apt install nvidia-utils-580-server # version 580.159.03-0ubuntu0.
26.04.1
sudo apt install nvidia-utils-595 # version 595.71.05-0ubuntu0.2
6.04.1
sudo apt install nvidia-utils-595-server # version 595.71.05-0ubuntu0.2
6.04.1
```

O comando lspci identificou o Microsoft Basic Render Driver como controlador 3D no ambiente WSL2. O comando nvidia-smi não está disponível no ambiente, não sendo identificada uma GPU NVIDIA utilizável pelo experimento.

6.6. Ollama

Versão: 0.34.2

Comando:

ollama --version

```
mariana@Lenovo:~$ ollama --version
ollama version is 0.34.2
```

6.7. Modelos instalados inicialmente

No início do experimento foi instalado o modelo llama3.2:3b, utilizado como modelo de inferência local nos experimentos.

Comando:

ollama list

```
mariana@Lenovo:~$ ollama list
NAME          ID          SIZE    MODIFIED
llama3.2:3b   a80c4f17acd5 2.0 GB  26 hours ago
```

6.8. API do Ollama

A API local foi testada por meio do comando:

Comando:

curl http://localhost:11434/api/tags

Resultado após a instalação do modelo:

```
mariana@Lenovo:~$ curl http://localhost:11434/api/tags
{"models":[{"name":"llama3.2:3b","model":"llama3.2:3b","modified_at":"2026-09-20T20:11:32.769222776-03:00","size":2019393189,"digest":"a80c4f17acd55265feec403c7aef86be0c25983ab279d83f3bcd3abbc5b8b72","details":{"parent_model":"","format":"gguf","family":"llama","families":["llama"],"parameter_size":"3.2B","quantization_level":"Q4_K_M","context_length":131072,"embedding_length":3072},"capabilities":["completion","tools"]}]}
```

A resposta da API local confirma que o modelo llama3.2:3b está registrado no serviço Ollama e pode ser acessado por meio da API HTTP local.

7. MODELO DE LINGUAGEM

7.1. Identificação

Nome completo: Meta Llama 3.2 3B Instruct

Família: Llama 3.2

Checkpoint/variante: Llama 3.2 3B Instruct — Q4_K_M

Parâmetros: 3,21 bilhões

URL do Hugging Face: <https://huggingface.co/meta-llama/Llama-3.2-3B-Instruct>

Licença: Llama 3.2 Community License Agreement

Formato: GGUF

Quantização: Q4_K_M

Tamanho do arquivo: aproximadamente 2,0 GB

Janela de contexto: 131.072 tokens (128K)

7.2. Justificativa da escolha

O Llama 3.2 3B foi escolhido por atender ao limite de até 10 bilhões de parâmetros estabelecido na atividade e por possuir uma versão quantizada Q4_K_M adequada à execução local por meio do Ollama. A escolha também considera as limitações do ambiente experimental, composto por CPU Intel Core i3-10110U, 4 threads e aproximadamente 5,7 GB de RAM disponíveis no WSL2, sem GPU NVIDIA disponível para aceleração. O modelo permite realizar inferência local e observar processos, threads, consumo de recursos, comunicação e chamadas de sistema, atendendo aos objetivos experimentais da atividade.

7.3. Adequação ao hardware

O Llama 3.2 3B foi considerado adequado ao hardware disponível para o experimento por possuir aproximadamente 3,21 bilhões de parâmetros e utilizar a quantização Q4_K_M, com tamanho aproximado de 2,0 GB. O ambiente possui CPU Intel Core i3-10110U, com 2 núcleos físicos e 4 threads, aproximadamente 5,7 GB de RAM disponíveis no WSL2 e 2,0

GB de swap. Não foi identificada uma GPU NVIDIA disponível para aceleração. A execução inicial do modelo foi realizada com sucesso pelo Ollama, confirmando sua viabilidade no ambiente. O impacto sobre CPU, memória, processos e threads será analisado durante os experimentos.

8. INSTALAÇÃO E CONFIGURAÇÃO

8.1. Instalação do WSL

O ambiente utilizado foi o WSL2, disponibilizado pelo Windows para execução de uma distribuição Linux integrada ao sistema.

Distribuição utilizada: Ubuntu 26.04.1 LTS

Ambiente: WSL2

A identificação do ambiente foi realizada por meio dos comandos:

```
uname -a
```

```
cat /etc/os-release
```

As saídas desses comandos estão apresentadas na seção **Ambiente Experimental** → **Sistema operacional**.

8.2. Instalação do Ollama

Versão utilizada: 0.34.2

O Ollama foi instalado no ambiente Ubuntu/WSL2 e posteriormente verificado por meio do comando:

```
ollama --version
```

A instalação e o funcionamento do serviço foram confirmados por meio da execução do modelo llama3.2:3b e do acesso à API local na porta 11434.

Comandos utilizados na configuração e verificação:

```
ollama --version
```

```
ollama pull llama3.2:3b
```

```
ollama list
```

```
ollama run llama3.2:3b
```

```
curl http://localhost:11434/api/tags
```

As evidências correspondentes encontram-se nas seções Ambiente Experimental, Modelo de Linguagem e Execução inicial do modelo.

8.3. Instalação do Open WebUI

Versão/commit: v0.11.4.

Método de instalação: uv com Python 3.11.

Comandos utilizados:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

```
source $HOME/.local/bin/env
```

```
uv --version
```

```
DATA_DIR=~/.open-webui uvx --python 3.11 open-webui@latest serve
```

Após a instalação, o funcionamento do servidor foi verificado por meio do comando:

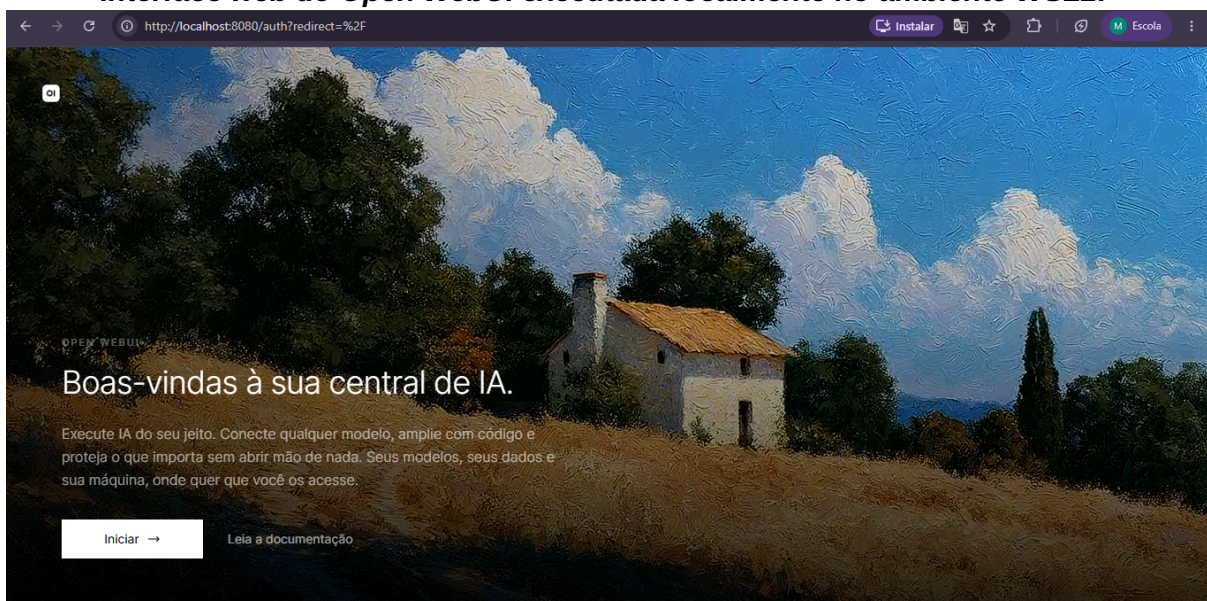
```
curl -I http://localhost:8080
```

Verificação do servidor local do Open WebUI por meio de requisição HTTP:

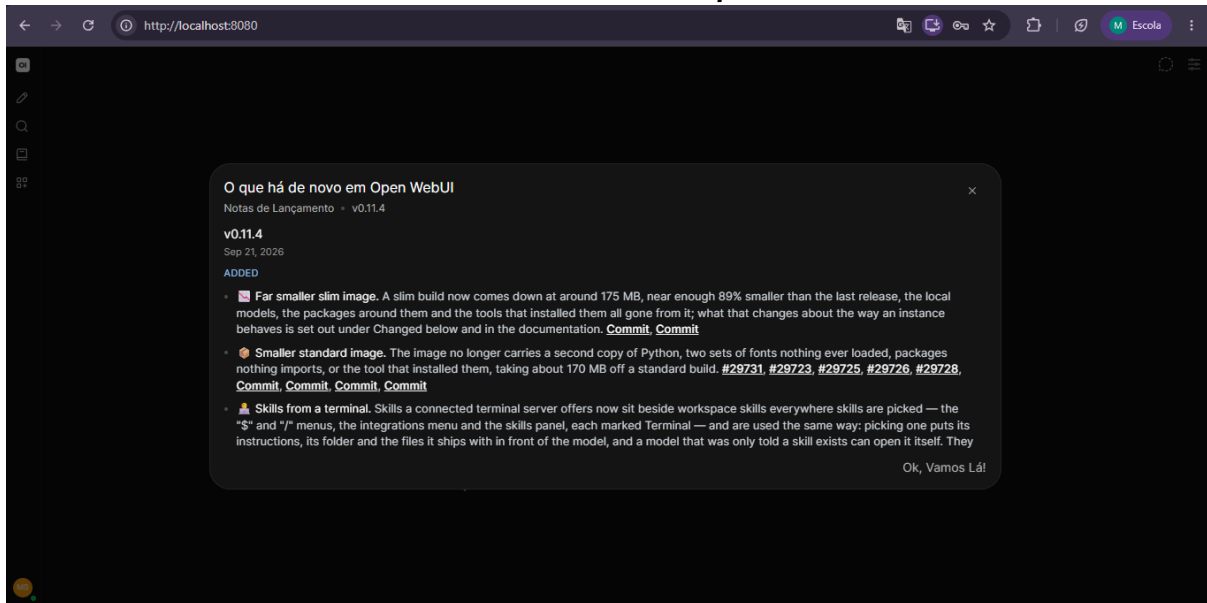
```
mariana@Lenovo:~$ curl -I http://localhost:8080
HTTP/1.1 200 OK
date: Tue, 22 Sep 2026 09:13:01 GMT
server: uvicorn
content-type: text/html; charset=utf-8
accept-ranges: bytes
content-length: 11318
last-modified: Tue, 22 Sep 2026 02:13:49 GMT
etag: "949b992542c3c1aba1d314736d0f3208"
x-process-time: 0.002885
```

A resposta obtida foi **HTTP/1.1 200 OK**, confirmando que o Open WebUI estava ativo e respondendo por HTTP na **porta local 8080**. O servidor identificado na resposta foi o Uvicorn. Em seguida, a interface web foi acessada pelo navegador por meio do endereço local **http://localhost:8080**, confirmando o funcionamento da camada de aplicação.

Interface web do Open WebUI executada localmente no ambiente WSL2:



Tela exibindo a versão do Open WebUI:



8.4. Adaptações realizadas

Até o momento, não foram realizadas alterações específicas na configuração padrão do Open WebUI. A aplicação foi instalada utilizando o procedimento de instalação local com uv e Python 3.11.

Eventuais alterações necessárias para conexão com o Ollama, armazenamento, portas, parâmetros de execução ou demais configurações serão registradas após a conclusão da configuração e dos testes da aplicação.

8.5. Erros encontrados

Até o momento, não foi identificado erro crítico que tenha impedido a instalação do Open WebUI.

Durante o carregamento do modelo auxiliar all-MiniLM-L6-v2, foi apresentada a mensagem `embeddings.position_ids | UNEXPECTED`. O próprio relatório de carregamento informou que essa ocorrência pode ser ignorada quando o modelo é carregado a partir de uma arquitetura ou tarefa diferente. Portanto, a mensagem foi tratada como um aviso de carregamento e não como falha da instalação.

Trecho do aviso:

```
026-09-21 23:39:18.029 | INFO | sentence_transformers.base.model:_load_modules:992 - Loading SentenceTransformer model from /home/mariana/.cache/huggingface/hub/models--sentence-transformers--all-MiniLM-L6-v2/snapshots/1110a243fdf4706b3f48f1d95db1a4f5529b4d41.
Download complete: ██████████ | 878MB, 7.72MB/s
Reconstruction complete: 100%|██████████| 931MB / 931MB, 13.8MB/s
Loading weights: 100%|██████████| 103/103 [00:00<00:00, 2593.88it/s]
BertModel LOAD REPORT from: /home/mariana/.cache/huggingface/hub/models--sentence-transformers--all-MiniLM-L6-v2/snapshots/1110a243fdf4706b3f48f1d95db1a4f5529b4d41
Key | Status | | 
-----+-----+---
embeddings.position_ids | UNEXPECTED | | 

Notes:
- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
2026-09-21 23:39:23.393 | INFO | open_webui.main:lifespan:387 - Installing external dependencies of functions and tools...
2026-09-21 23:39:23.408 | INFO | open_webui.utils.plugin:install_frontend_matter_requirements:450 - No requirements found in frontmatter.
2026-09-21 23:39:23.410 | INFO | open_webui.utils.automations:schedule
```

A confirmação definitiva do funcionamento do servidor será realizada por meio de acesso à interface web local.

9. EXECUÇÃO INICIAL

9.1. Download do modelo

Modelo: llama3.2:3b

Comando: ollama pull llama3.2:3b

Horário de início: não registrado durante o download original

Horário de término: não registrado durante o download original

Tempo de download: não registrado

Espaço ocupado:1,9 GB, conforme medição realizada com du -sh
/usr/share/ollama/.ollama/models.

A instalação do modelo foi confirmada por meio do comando `ollama list`, que apresentou o modelo llama3.2:3b com tamanho aproximado de 2,0 GB. A medição direta do armazenamento indicou 1,9 GB ocupados pelos arquivos do modelo em `/usr/share/ollama/ollama/models`, sendo aproximadamente 1,9 GB correspondentes ao diretório blobs.

```
mariana@Lenovo: ~  
mariana@Lenovo:~$ ollama list  
NAME          ID          SIZE      MODIFIED  
llama3.2:3b    a80c4f17acd5 2.0 GB    46 hours ago  
mariana@Lenovo:~$ du -sh /usr/share/ollama/.ollama/models/blobs  
1.9G    /usr/share/ollama/.ollama/models/blobs  
mariana@Lenovo:~$ du -sh /usr/share/ollama/.ollama/models  
1.9G    /usr/share/ollama/.ollama/models  
mariana@Lenovo:~$ |
```

9.2. Primeira execução

Horário de início: 17:47:59

Horário de término: 17:49:51

Comando: `time ollama run llama3.2:3b`

Prompt utilizado: “Explique em poucas frases o que é um processo em um sistema operacional e qual é a diferença entre processo e thread.”

Primeira execução do modelo utilizado:

```
mariana@Lenovo:~$ date '+%Y-%m-%d %H:%M:%S'  
2026-09-22 17:47:59  
mariana@Lenovo:~$ time ollama run llama3.2:3b  
>>> Explique em poucas frases o que é um processo em um sistema operacio  
... nal e qual é a diferença entre processo e thread.  
Um processo em um sistema operacional é uma unidade de execução de  
um programa que tem recursos próprios, como memória e recursos de  
sistema. Já uma thread é uma unidade de execução dentro de um  
processo que permite realizar tarefas paralelas sem que o processo  
se divida.  
  
Em outras palavras:  
  
- Processo: uma unidade de execução completa  
- Thread: uma unidade de execução dentro de um processo  
  
>>> /bye  
  
real    1m37.597s  
user    0m0.170s  
sys     0m0.110s  
mariana@Lenovo:~$ date '+%Y-%m-%d %H:%M:%S'  
2026-09-22 17:49:51
```

Tempo de resposta: 1m37.597s, conforme a medição apresentada pelo comando `time`.

Observações:

O modelo foi executado localmente com sucesso por meio do Ollama e respondeu ao prompt sem apresentar erro. A resposta diferenciou processo e thread de forma direta, caracterizando o processo como uma unidade de execução com seus próprios recursos e a thread como uma unidade de execução pertencente a um processo.

Durante a execução, foi utilizado o modelo llama3.2:3b, cuja configuração identificada pelo comando `ollama show` apresenta aproximadamente 3,2 bilhões de parâmetros, contexto de 131.072 tokens e quantização Q4_K_M.

10. PROCESSOS E THREADS

10.1. Processos observados

A análise de processos e threads foi realizada em dois momentos: antes da execução de uma requisição pelo Open WebUI e durante o processamento do modelo pelo Ollama. Foram utilizados os comandos `ps`, `ps -eLf`, `pstree -p`, `htop` e `top -H`.

10.2. Situação antes da execução do open webui

Antes de enviar uma requisição pelo Open WebUI, foi realizada uma coleta dos processos em execução no ambiente WSL2. O processo principal do Ollama foi identificado pelo PID 148, com PPID 1, apresentando inicialmente 11 threads.

Nesse momento, o processo do Ollama apresentava baixa utilização de CPU, indicando que o servidor estava em execução, mas não estava realizando uma inferência ativa.

Foram utilizados os comandos:

```
ps -eo pid,ppid,stat,ni,pri,psr,pcpu,pmem,nlwp,comm --sort=-pcpu
```

```
ps -eLf
```

```
pstree -p
```

```
htop
```

```
top -H
```

As saídas completas dessas ferramentas foram registradas por meio de capturas de tela e são apresentadas nos anexos como evidências da situação inicial do ambiente.

Comando: `ps -eo pid,ppid,stat,ni,pri,psr,pcpu,pmem,nlwp,comm --sort=-pcpu`

```
mariana@Lenovo:~$ ps -eo pid,ppid,stat,ni,pri,psr,pcpu,pmem,nlwp,comm,args --sort=-pcpu | grep -E 'ollama|open-webui' | grep -v grep
148      1 Ssl  0 19  0 0.0 0.5 10 ollama /usr/local/bin/ollama serve
```

Comando: `ps -eLf`

```

mariana@Lenovo:~$ ps -eLf
UID      PID     PPID    LWP   C  NLWP  STIME TTY          TIME CMD
root         1         0        1   0    1 17:38 ?          00:00:01 /sbin/i
root         2         1        2   0    2 17:38 hvc0      00:00:00 /init
root         2         1        9   0    2 17:38 hvc0      00:00:00 /init
root         7         2        7   0    2 17:38 hvc0      00:00:00 plan9 -
root         7         2        8   0    2 17:38 hvc0      00:00:00 plan9 -
root        53         1       53   0    1 17:38 ?          00:00:00 /usr/li
systemd+   88         1       88   0    1 17:38 ?          00:00:00 /usr/li
root       97         1       97   0    1 17:38 ?          00:00:02 /usr/li
root      117         1     117   0    1 17:38 ?          00:00:00 /bin/sh
root      118         1     118   0    1 17:38 ?          00:00:00 /usr/sb
message+  119         1     119   0    1 17:38 ?          00:00:00 @dbus-d
root     147         1     147   0    1 17:38 ?          00:00:00 /usr/bi
ollama   148         1     148   0   10 17:38 ?          00:00:00 /usr/lo
ollama   148         1     192   0   10 17:38 ?          00:00:00 /usr/lo
ollama   148         1     193   0   10 17:38 ?          00:00:00 /usr/lo
ollama   148         1     194   0   10 17:38 ?          00:00:00 /usr/lo
ollama   148         1     195   0   10 17:38 ?          00:00:00 /usr/lo
ollama   148         1     199   0   10 17:38 ?          00:00:00 /usr/lo
ollama   148         1     200   0   10 17:38 ?          00:00:00 /usr/lo
ollama   148         1     204   0   10 17:38 ?          00:00:00 /usr/lo
ollama   148         1     205   0   10 17:38 ?          00:00:00 /usr/lo
ollama   148         1     265   0   10 17:38 ?          00:00:00 /usr/lo
root     156         1     156   0    1 17:38 ?          00:00:00 /usr/li
syslog   168         1     168   0    4 17:38 ?          00:00:00 /usr/sb
syslog   168         1     238   0    4 17:38 ?          00:00:00 /usr/sb
syslog   168         1     239   0    4 17:38 ?          00:00:00 /usr/sb
syslog   168         1     240   0    4 17:38 ?          00:00:00 /usr/sb
_chrony  281       117     281   0    1 17:38 ?          00:00:00 /usr/sb
_chrony  296       281     296   0    1 17:38 ?          00:00:00 /usr/sb
root     302         1     302   0    2 17:38 ?          00:00:00 /usr/bi
root     302         1     324   0    2 17:38 ?          00:00:00 /usr/bi
root     306         1     306   0    1 17:38 tty1       00:00:00 /usr/sb
root     382         2     382   0    1 17:38 ?          00:00:00 login -
mariana  432         1     432   0    1 17:38 ?          00:00:00 /usr/li
mariana  434       432     434   0    1 17:38 ?          00:00:00 (sd-pam
mariana  459       382     459   0    1 17:38 pts/1      00:00:00 -bash
root     965         2     965   0    1 18:04 ?          00:00:00 /init
root     967       965     967   0    1 18:04 ?          00:00:00 /init
mariana  971       967     971   0    1 18:04 pts/0      00:00:00 -bash
root    1085         1    1085   0    8 18:09 ?          00:00:00 /usr/li
root    1085         1    1086   0    8 18:09 ?          00:00:00 /usr/li
root    1085         1    1087   0    8 18:09 ?          00:00:00 /usr/li
root    1085         1    1088   0    8 18:09 ?          00:00:00 /usr/li
root    1085         1    1089   0    8 18:09 ?          00:00:00 /usr/li
root    1085         1    1090   0    8 18:09 ?          00:00:00 /usr/li
root    1085         1    1091   0    8 18:09 ?          00:00:00 /usr/li
root    1085         1    1094   0    8 18:09 ?          00:00:00 /usr/li
root    1189        97    1189   0    1 18:15 ?          00:00:00 (udev-w
root    1190        97    1190   0    1 18:15 ?          00:00:00 (udev-w
mariana 1196       971    1196   0    1 18:15 pts/0      00:00:00 ps -eLf

```

Comando: pstree -p


```
0[ ||| 2.6%] Tasks: 29, 25 thr, 0 kthr; 1 running
1[ | 2.0%] Load average: 0.00 0.02 0.04
2[ | 0.0%] Uptime: 01:13:14
3[ | 0.7%]
Mem[ ||||| 386M/5.71G]
Swp[ | 6M/2.00G]

Main I/O
PID USER PRI NI VIRT RES SHR S CPU% MEM% TIME+ Command
265 ollama 20 0 1912M 38468 0 S 0.0 0.5 0:00.16 /usr/local/bin/ollama serve
281 _chrony 20 0 28424 6384 5604 S 0.0 0.1 0:00.25 /usr/sbin/chronyd -n -F 1 -x
296 _chrony 20 0 12096 2440 1792 S 0.0 0.0 0:00.01 /usr/sbin/chronyd -n -F 1 -x
382 root 20 0 120M 32656 18048 S 0.0 0.5 0:00.28 /usr/bin/python3 /usr/share/unattended-upgrades/unattended-upgrade-shutdown --wai
306 root 20 0 5492 2780 2592 S 0.0 0.0 0:00.02 /usr/sbin/agetty --noreset --noclear --issue-file=/etc/issue:/etc/issue.d/run/is
324 root 20 0 120M 32656 0 S 0.0 0.5 0:00.00 /usr/bin/python3 /usr/share/unattended-upgrades/unattended-upgrade-shutdown --wai
382 root 20 0 8648 5636 4792 S 0.0 0.1 0:00.02 login -- mariana
432 mariana 20 0 22520 13404 10956 S 0.0 0.2 0:00.25 /usr/lib/systemd/systemd --user
434 mariana 20 0 22916 4088 2088 S 0.0 0.1 0:00.00 (sd-pam)
459 mariana 20 0 6136 5468 3928 S 0.0 0.1 0:00.05 -bash
965 root 20 0 3196 1124 980 S 0.0 0.0 0:00.00 /init
967 root 20 0 3196 1256 1108 S 0.0 0.0 0:00.34 /init
971 mariana 20 0 6268 5608 3936 S 0.0 0.1 0:00.28 -bash
1585 root 20 0 1758M 14472 11904 S 0.0 0.2 0:00.47 /usr/libexec/wsl-pro-service
1586 root 20 0 1758M 14472 0 S 0.0 0.2 0:00.01 /usr/libexec/wsl-pro-service
1587 root 20 0 1758M 14472 0 S 0.0 0.2 0:00.00 /usr/libexec/wsl-pro-service
1588 root 20 0 1758M 14472 0 S 0.0 0.2 0:00.00 /usr/libexec/wsl-pro-service
1589 root 20 0 1758M 14472 0 S 0.0 0.2 0:00.00 /usr/libexec/wsl-pro-service
1590 root 20 0 1758M 14472 0 S 0.0 0.2 0:00.00 /usr/libexec/wsl-pro-service
1591 root 20 0 1758M 14472 0 S 0.0 0.2 0:00.01 /usr/libexec/wsl-pro-service
1598 root 20 0 1758M 14472 0 S 0.0 0.2 0:00.00 /usr/libexec/wsl-pro-service
1737 polkitd 20 0 299M 8568 7756 S 0.0 0.1 0:00.09 /usr/lib/polkit-1/polkitd --no-debug --log-level=notice
1739 polkitd 20 0 299M 8568 0 S 0.0 0.1 0:00.01 /usr/lib/polkit-1/polkitd --no-debug --log-level=notice
1740 polkitd 20 0 299M 8568 0 S 0.0 0.1 0:00.00 /usr/lib/polkit-1/polkitd --no-debug --log-level=notice
1741 polkitd 20 0 299M 8568 0 S 0.0 0.1 0:00.00 /usr/lib/polkit-1/polkitd --no-debug --log-level=notice
1825 mariana 20 0 5992 4932 3908 R 1.3 0.1 0:00.32 htop
1831 root 20 0 35256 6284 3284 S 0.0 0.1 0:00.00 (udev-worker)
1832 root 20 0 35256 6284 3284 S 0.0 0.1 0:00.00 (udev-worker)
F1Help F2Setup F3Search F4Filter F5Tree F6SortBy F7Nice F8Nice F9Kill F10Quit
```

Comando: top -H

```
mariana@Lenovo:~$ top -H
top - 18:52:00 up 1:13, 1 user, load average: 0.00, 0.02, 0.03
Threads: 44 total, 1 running, 43 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.0 us, 0.0 sy, 0.0 ni,100.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 5842.6 total, 5189.3 free, 555.5 used, 285.0 buff/cache
MiB Swap: 2048.0 total, 2048.0 free, 0.0 used. 5287.1 avail Mem

  PID USER      PR  NI  VIRT  RES  SHR S %CPU  %MEM    TIME+  COMMAND
 1847 mariana   20   0   8340  6096  3900 R   0.3   0.1   0:00.03 top
    1 root      20   0  24160 15384 11564 S   0.0   0.3   0:02.06 systemd
    2 root      20   0   3180  2208  2072 S   0.0   0.0   0:00.07 init-systemd(Ub
    9 root      20   0   3180  2208  2072 S   0.0   0.0   0:00.00 Interop
    7 root      20   0   3180  2044  1940 S   0.0   0.0   0:00.00 init
    8 root      20   0   3180  2044  1940 S   0.0   0.0   0:00.00 init
   53 root      19  -1  50400 16944 15628 S   0.0   0.3   0:00.56 systemd-journal
   88 systemd+  20   0  22416 14516 12028 S   0.0   0.2   0:00.18 systemd-resolve
   97 root      20   0  35252 12092  9128 S   0.0   0.2   0:05.65 systemd-udev
  117 root      20   0   2888  2008  1884 S   0.0   0.0   0:00.06 chronyd-starter
  118 root      20   0   4472  3148  2900 S   0.0   0.1   0:00.04 cron
  119 message+  20   0   8824  5440  4740 S   0.0   0.1   0:00.11 dbus-daemon
  147 root      20   0   44096 29744 14608 S   0.0   0.5   0:00.32 networkd-dispat
  148 ollama    20   0 1958540 30468 19988 S   0.0   0.5   0:00.09 ollama
  192 ollama    20   0 1958540 30468 19988 S   0.0   0.5   0:00.10 ollama
  193 ollama    20   0 1958540 30468 19988 S   0.0   0.5   0:00.21 ollama
  194 ollama    20   0 1958540 30468 19988 S   0.0   0.5   0:00.00 ollama
  195 ollama    20   0 1958540 30468 19988 S   0.0   0.5   0:00.00 ollama
  199 ollama    20   0 1958540 30468 19988 S   0.0   0.5   0:00.31 ollama
  200 ollama    20   0 1958540 30468 19988 S   0.0   0.5   0:00.11 ollama
  204 ollama    20   0 1958540 30468 19988 S   0.0   0.5   0:00.13 ollama
  205 ollama    20   0 1958540 30468 19988 S   0.0   0.5   0:00.10 ollama
  265 ollama    20   0 1958540 30468 19988 S   0.0   0.5   0:00.16 ollama
  156 root      20   0   18436  9408  8168 S   0.0   0.2   0:00.20 systemd-logind
  168 syslog    20   0  220548  5996  4668 S   0.0   0.1   0:00.10 rsyslogd
  238 syslog    20   0  220548  5996  4668 S   0.0   0.1   0:00.04 in:imuxsock
  239 syslog    20   0  220548  5996  4668 S   0.0   0.1   0:00.01 in:imklog
  240 syslog    20   0  220548  5996  4668 S   0.0   0.1   0:00.02 rs:main Q:Reg
  281 _chrony     20   0   20424  6384  5604 S   0.0   0.1   0:00.25 chronyd
  296 _chrony     20   0   12096  2440  1792 S   0.0   0.0   0:00.01 chronyd
  302 root      20   0 123460 32656 18048 S   0.0   0.5   0:00.28 unattended-upgr
  324 root      20   0 123460 32656 18048 S   0.0   0.5   0:00.00 gmain
  306 root      20   0   5492  2780  2592 S   0.0   0.0   0:00.02 agetty
  382 root      20   0   8648  5636  4792 S   0.0   0.1   0:00.02 login
  432 mariana   20   0  22520 13404 10956 S   0.0   0.2   0:00.25 systemd
  434 mariana   20   0  22916  4088  2088 S   0.0   0.1   0:00.00 (sd-pam)
  459 mariana   20   0   6136  5468  3928 S   0.0   0.1   0:00.05 bash
  965 root      20   0   3196  1124   980 S   0.0   0.0   0:00.00 SessionLeader
  967 root      20   0   3196  1256  1108 S   0.0   0.0   0:00.35 Relay(971)
  971 mariana   20   0   6268  5608  3936 S   0.0   0.1   0:00.28 bash
 1737 polkitd    20   0 306444  8568  7756 S   0.0   0.1   0:00.09 polkitd
 1739 polkitd    20   0 306444  8568  7756 S   0.0   0.1   0:00.02 gmain
 1740 polkitd    20   0 306444  8568  7756 S   0.0   0.1   0:00.00 pool-spawner
 1741 polkitd    20   0 306444  8568  7756 S   0.0   0.1   0:00.00 gdbus
```

10.3. Situação durante a execução do Open WebUI

Após o envio de uma requisição pelo Open WebUI, foi realizada uma nova coleta dos processos. Nesse momento, foi possível observar uma alteração significativa na utilização dos recursos.

O comando ps identificou o processo llama-server, de PID 2091, associado ao processo principal do Ollama (PID 148). Esse processo apresentava 11 threads, utilização de aproximadamente 111% de CPU e 52,5% de memória, conforme a coleta realizada:

PID	PPID	STAT	NI	PRI	PSR	CPU	%MEM	NLWP	COM MAND
2091	148	Sl	0	19	0	111	52.5	11	llama-

									server
1973	1915	Sl+	0	19	2	2.8	19.7	30	python
148	1	Ssl	0	19	0	0.0	0.5	11	ollama

O processo python, de **PID 1973**, corresponde à execução do Open WebUI, iniciada por meio do uv, e também apresentou atividade durante a coleta. Assim, foi possível observar na mesma captura a presença das duas camadas principais da aplicação: o Open WebUI e o servidor de inferência do Ollama.

Comando: ps -eo pid,ppid,stat,ni,pri,psr,pcpu,pmem,nlwp,comm --sort=-pcpu

```

mariana@Lenovo:~$ ps -eo pid,ppid,stat,ni,pri,psr,pcpu,pmem,nlwp,comm,args --sort=-pcpu
  PID   PPID  STAT  NI  PRI  PSR  %CPU  %MEM  NLWP  COMMAND  COMMAND
  2091   148  Sl     0   19   0   111  52.5   11  llama-server /usr/local/lib/ol
  1973   1915 Sl+    0   19   2   2.8  19.7   30  python /home/mariana/.ca
    97    1  Ss     0   19   1   0.1   0.1    1  systemd-udev /usr/lib/systemd/
  1915   1889 Sl+    0   19   2   0.1   0.6    2  uv /home/mariana/.lo
  2237    1  Ssl    0   19   0   0.0   0.2    8  wsl-pro-service /usr/libexec/wsl-
  148    1  Ssl    0   19   0   0.0   0.5   11  ollama /usr/local/bin/ol
    1    0  Ss     0   19   0   0.0   0.2    1  systemd /sbin/init
    53    1  S<s    -1  20   2   0.0   0.2    1  systemd-journal /usr/lib/systemd/
   967   965  S      0   19   2   0.0   0.0    1  Relay(971) /init
  1737    1  Ssl    0   19   1   0.0   0.1    4  polkitd /usr/lib/polkit-1
   971   967  Ss     0   19   0   0.0   0.0    1  bash -bash
  1889   1882 Ss     0   19   3   0.0   0.0    1  bash -bash
   147    1  Ss     0   19   0   0.0   0.4    1  networkd-dispat /usr/bin/python3
   302    1  Ssl    0   19   2   0.0   0.5    2  unattended-upgr /usr/bin/python3
   281   117  S      0   19   0   0.0   0.1    1  chronyd /usr/sbin/chronyd
   432    1  Ss     0   19   0   0.0   0.2    1  systemd /usr/lib/systemd/
   168    1  Ssl    0   19   2   0.0   0.1    4  rsyslogd /usr/sbin/rsyslog
   156    1  Ss     0   19   2   0.0   0.1    1  systemd-logind /usr/lib/systemd/
    88    1  Ss     0   19   3   0.0   0.2    1  systemd-resolve /usr/lib/systemd/
   119    1  Ss     0   19   3   0.0   0.0    1  dbus-daemon @dbus-daemon --sy
  1882   1881 S      0   19   1   0.0   0.0    1  Relay(1889) /init
    2    1  Sl+    0   19   3   0.0   0.0    2  init-systemd(Ub /init
   117    1  Ss     0   19   0   0.0   0.0    1  chronyd-starter /bin/sh /usr/lib/
   459   382  Ss+    0   19   2   0.0   0.0    1  bash -bash
   118    1  Ss     0   19   2   0.0   0.0    1  cron /usr/sbin/cron -f
   382    2  Ss     0   19   3   0.0   0.0    1  login login -- mariana
   306    1  Ss+    0   19   0   0.0   0.0    1  agetty /usr/sbin/agetty
   296   281  S      0   19   3   0.0   0.0    1  chronyd /usr/sbin/chronyd
    7    2  Sl+    0   19   3   0.0   0.0    2  init plan9 --control-s
   434   432  S      0   19   0   0.0   0.0    1 (sd-pam) (sd-pam)
   965    2  Ss     0   19   3   0.0   0.0    1 SessionLeader /init
  1881    2  Ss     0   19   3   0.0   0.0    1 SessionLeader /init
  2385   971  R+     0   19   1   0.0   0.0    1 ps ps -eo pid,ppid,s

```

A análise detalhada com `ps -eLf` mostrou que o processo llama-server (PID 2091) possuía 11 threads. Duas delas apresentavam atividade de CPU significativamente maior:

LWP	PRI	NLWP	TIME
2091	51	11	00:07:01
2103	50	11	00:06:38

Comando: ps -eLf

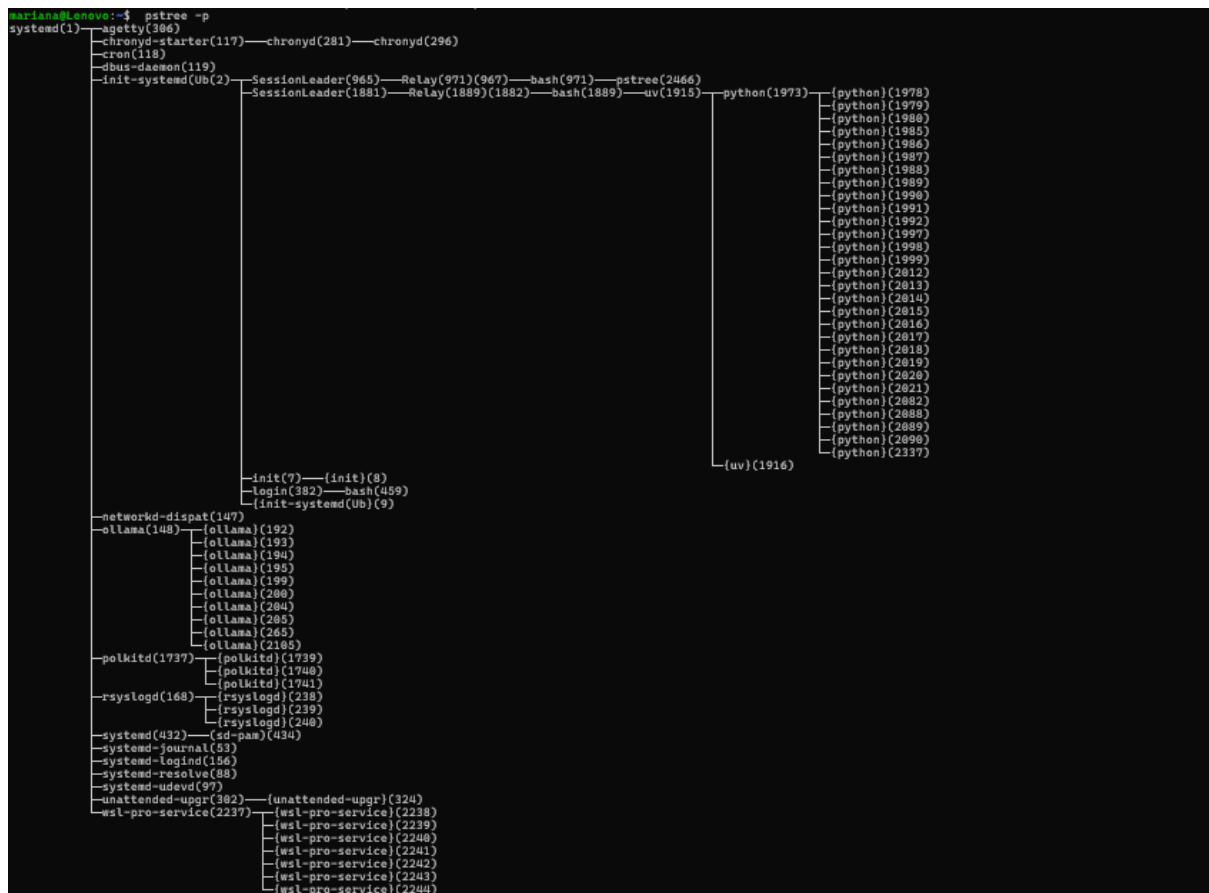
Trecho mais relevante da coleta:

ollama	2091	148	2091	51	11	19:04	?	00:07:01	/usr/lo
ollama	2091	148	2093	0	11	19:04	?	00:00:00	/usr/lo
ollama	2091	148	2094	0	11	19:04	?	00:00:00	/usr/lo
ollama	2091	148	2095	0	11	19:04	?	00:00:00	/usr/lo
ollama	2091	148	2096	0	11	19:04	?	00:00:00	/usr/lo
ollama	2091	148	2097	0	11	19:04	?	00:00:00	/usr/lo
ollama	2091	148	2098	0	11	19:04	?	00:00:00	/usr/lo
ollama	2091	148	2099	0	11	19:04	?	00:00:00	/usr/lo
ollama	2091	148	2100	0	11	19:04	?	00:00:00	/usr/lo
ollama	2091	148	2103	50	11	19:04	?	00:06:38	/usr/lo
ollama	2091	148	2104	0	11	19:04	?	00:00:00	/usr/lo

As demais threads do processo apresentaram tempo de CPU acumulado igual a zero no instante da coleta. Isso demonstra que as threads não necessariamente possuem a mesma carga de processamento durante a inferência.

A árvore de processos obtida com *ps tree -p* também mostrou o processo principal do Ollama:

Comando: *ps tree -p*



Além disso, a árvore evidenciou a cadeia de execução do Open WebUI, representada pelo uv e pelo processo Python, juntamente com suas respectivas threads.

Comando: *htop*

```
0[|||] 2.6% Tasks: 33, 49 thr, 0 kthr; 1 running
1[|||] 1.3% Load average: 0.01 0.24 0.36
2[|||] 1.3% Uptime: 01:45:01
3[|||] 1.3%
Mem[|||||] 1.13G/5.71G
Swap[|||||] 100K/2.00G

Main I/O
PID USER PRI NI VIRT RES SHR S CPU% MEM% TIME+ Command
1 root 20 0 24160 15384 11504 S 0.7 0.3 0:02.27 /sbin/init
2 root 20 0 3180 2208 2072 S 0.0 0.0 0:00.07 /init
7 root 20 0 3180 2044 1940 S 0.0 0.0 0:00.00 plan9 --control-socket 7 --log-level 4 --server-fd 8 --pipe-fd 10 --log-truncate
8 root 20 0 3180 2044 0 S 0.0 0.0 0:00.00 plan9 --control-socket 7 --log-level 4 --server-fd 8 --pipe-fd 10 --log-truncate
9 root 20 0 3180 2208 0 S 0.0 0.0 0:00.00 /init
53 root 19 -1 50400 17138 15800 S 0.0 0.3 0:01.10 /usr/lib/systemd/systemd-journald
88 systemd-re 20 0 22416 14516 12028 S 0.0 0.2 0:00.19 /usr/lib/systemd/systemd-resolved
97 root 20 0 35252 11912 8948 S 0.7 0.2 0:07.60 /usr/lib/systemd/systemd-udev
117 root 20 0 2888 2068 1884 S 0.0 0.0 0:00.06 /bin/sh /usr/lib/systemd/scripts/chronyd-starter.sh -n -F 1
118 root 20 0 4472 3148 2908 S 0.0 0.1 0:00.06 /usr/sbin/cron -F -p
119 messagebus 20 0 8524 5392 4692 S 0.0 0.1 0:00.14 dbus-daemon --system --address=systemd: --nofork --nopidfile --systemd-activation --syslog-only
147 root 20 0 44096 29572 14436 S 0.0 0.5 0:00.32 /usr/bin/python3 /usr/bin/networkd-dispatcher --run-startup-triggers
148 ollama 20 0 1984M 30880 19976 S 0.0 0.5 0:00.09 /usr/local/bin/ollama serve
156 root 20 0 18436 9408 8168 S 0.0 0.2 0:00.22 /usr/lib/systemd/systemd-logind
168 syslog 20 0 215M 6000 4668 S 0.0 0.1 0:00.05 /usr/sbin/rsyslogd -n -iNONE
192 ollama 20 0 1984M 30880 0 S 0.0 0.5 0:00.19 /usr/local/bin/ollama serve
193 ollama 20 0 1984M 30880 0 S 0.0 0.5 0:00.30 /usr/local/bin/ollama serve
194 ollama 20 0 1984M 30880 0 S 0.0 0.5 0:00.00 /usr/local/bin/ollama serve
195 ollama 20 0 1984M 30880 0 S 0.0 0.5 0:00.00 /usr/local/bin/ollama serve
199 ollama 20 0 1984M 30880 0 S 0.0 0.5 0:00.00 /usr/local/bin/ollama serve
200 ollama 20 0 1984M 30880 0 S 0.0 0.5 0:00.69 /usr/local/bin/ollama serve
204 ollama 20 0 1984M 30880 0 S 0.0 0.5 0:00.65 /usr/local/bin/ollama serve
205 ollama 20 0 1984M 30880 0 S 0.0 0.5 0:00.18 /usr/local/bin/ollama serve
238 syslog 20 0 215M 6000 0 S 0.0 0.1 0:00.05 /usr/sbin/rsyslogd -n -iNONE
239 syslog 20 0 215M 6000 0 S 0.0 0.1 0:00.01 /usr/sbin/rsyslogd -n -iNONE
240 syslog 20 0 215M 6000 0 S 0.0 0.1 0:00.04 /usr/sbin/rsyslogd -n -iNONE
265 ollama 20 0 1984M 30880 0 S 0.0 0.5 0:00.18 /usr/local/bin/ollama serve
281 chrony 20 0 20424 6384 5604 S 0.0 0.1 0:00.27 /usr/sbin/chronyd -n -F 1 -x
296 chrony 20 0 12096 2400 1992 S 0.0 0.0 0:00.01 /usr/sbin/chronyd -n -F 1 -x
302 root 20 0 1208 32468 17860 S 0.0 0.5 0:00.28 /usr/bin/python3 /usr/share/unattended-upgrades/unattended-upgrade-shutdown --wait-for-signal
306 root 20 0 5492 2780 2592 S 0.0 0.0 0:00.02 /usr/sbin/agetty --noreast --noclear --issue-file=/etc/issue:/etc/issue.d/run:/etc/issue.d:/usr/lib/issue.d --linux
324 root 20 0 1208 32468 0 S 0.0 0.5 0:00.00 /usr/bin/python3 /usr/share/unattended-upgrades/unattended-upgrade-shutdown --wait-for-signal
382 root 20 0 8640 5636 4792 S 0.0 0.1 0:00.02 Login: mariana
432 mariana 20 0 22520 13404 10956 S 0.0 0.2 0:00.25 /usr/lib/systemd/systemd --user
434 mariana 20 0 22916 4008 2088 S 0.0 0.1 0:00.00 (sd-pan)
459 mariana 20 0 6136 4836 3296 S 0.0 0.1 0:00.05 -bash
965 root 20 0 3196 1116 980 S 0.0 0.0 0:00.00 /init

Help F2Setup F3Search F4Filter F5Free F6SortBy F7JFile F8JFile F9JFile F10JFile F11JFile F12JFile
```

```
0[|||] 2.1% Tasks: 33, 49 thr, 0 kthr; 1 running
1[|||] 1.3% Load average: 0.07 0.21 0.36
2[|||] 1.8% Uptime: 01:46:01
3[|||] 1.1%
Mem[|||||] 1.13G/5.71G
Swap[|||||] 100K/2.00G

Main I/O
PID USER PRI NI VIRT RES SHR S CPU% MEM% TIME+ Command
1889 mariana 20 0 6268 4996 3320 S 0.0 0.1 0:00.07 -bash
1915 mariana 20 0 1351M 40280 26792 S 0.0 0.7 0:00.02 /home/mariana/.local/bin/uv tool uvx --python 3.11 open-webui@latest serve
1916 mariana 20 0 1351M 40280 0 S 0.0 0.7 0:00.44 /home/mariana/.local/bin/uv tool uvx --python 3.11 open-webui@latest serve
1973 mariana 20 0 6280M 1151M 409M S 1.1 19.7 0:35.80 /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/python /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/open
1978 mariana 20 0 6280M 1151M 0 S 0.0 0.7 0:00.10 /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/python /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/open
1979 mariana 20 0 6280M 1151M 0 S 0.0 0.7 0:00.10 /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/python /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/open
1980 mariana 20 0 6280M 1151M 0 S 0.0 0.7 0:00.10 /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/python /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/open
1985 mariana 20 0 6280M 1151M 0 S 0.0 0.7 0:00.09 /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/python /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/open
1986 mariana 20 0 6280M 1151M 0 S 0.0 0.7 0:00.10 /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/python /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/open
1987 mariana 20 0 6280M 1151M 0 S 0.0 0.7 0:00.10 /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/python /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/open
1988 mariana 20 0 6280M 1151M 0 S 0.0 0.7 0:00.00 /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/python /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/open
1989 mariana 20 0 6280M 1151M 0 S 0.0 0.7 0:00.00 /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/python /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/open
1990 mariana 20 0 6280M 1151M 0 S 0.0 0.7 0:00.00 /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/python /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/open
1991 mariana 20 0 6280M 1151M 0 S 0.0 0.7 0:00.00 /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/python /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/open
1992 mariana 20 0 6280M 1151M 0 S 0.0 0.7 0:00.00 /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/python /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/open
1997 mariana 20 0 6280M 1151M 0 S 0.0 0.7 0:00.00 /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/python /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/open
1998 mariana 20 0 6280M 1151M 0 S 0.0 0.7 0:00.37 /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/python /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/open
1999 mariana 20 0 6280M 1151M 0 S 0.0 0.7 0:00.04 /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/python /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/open
2012 mariana 20 0 6280M 1151M 0 S 0.0 0.7 0:00.06 /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/python /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/open
2013 mariana 20 0 6280M 1151M 0 S 0.0 0.7 0:00.36 /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/python /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/open
2014 mariana 20 0 6280M 1151M 0 S 0.0 0.7 0:00.34 /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/python /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/open
2015 mariana 20 0 6280M 1151M 0 S 0.0 0.7 0:00.00 /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/python /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/open
2016 mariana 20 0 6280M 1151M 0 S 0.0 0.7 0:00.36 /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/python /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/open
2017 mariana 20 0 6280M 1151M 0 S 0.0 0.7 0:00.34 /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/python /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/open
2018 mariana 20 0 6280M 1151M 0 S 0.0 0.7 0:00.00 /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/python /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/open
2019 mariana 20 0 6280M 1151M 0 S 0.0 0.7 0:00.00 /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/python /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/open
2020 mariana 20 0 6280M 1151M 0 S 0.0 0.7 0:00.00 /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/python /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/open
2021 mariana 20 0 6280M 1151M 0 S 0.0 0.7 0:00.00 /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/python /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/open
2082 mariana 20 0 6280M 1151M 0 S 0.0 0.7 0:00.36 /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/python /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/open
2088 mariana 20 0 6280M 1151M 0 S 0.0 0.7 0:00.18 /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/python /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/open
2089 mariana 20 0 6280M 1151M 0 S 0.0 0.7 0:00.18 /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/python /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/open
2090 mariana 20 0 6280M 1151M 0 S 0.0 0.7 0:00.19 /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/python /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/open
2105 ollama 20 0 1984M 30880 0 S 0.0 0.5 0:00.05 /usr/local/bin/ollama serve
2137 mariana 20 0 6280M 1151M 0 S 0.0 0.7 0:00.00 /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/python /home/mariana/.cache/uv/archive-v0/bVPrse18C1y7yrVH/bin/open
2498 mariana 20 0 6504 5212 3676 R 2.2 0.1 0:00.98 htop
2505 root 20 0 35256 6268 3268 S 0.0 0.1 0:00.02 (udev-worker)
2506 root 20 0 35256 6268 3268 S 0.0 0.1 0:00.02 (udev-worker)

Help F2Setup F3Search F4Filter F5Free F6SortBy F7JFile F8JFile F9JFile F10JFile F11JFile F12JFile
```

Durante a espera pela resposta, o top -H foi mantido em execução para observar as threads individualmente. A captura realizada durante a inferência constitui evidência da atividade de processamento do modelo e da utilização dos recursos computacionais.

Comando: top -H

Durante a espera da resposta:

```
mariana@mariana:~$ top -H
top - 19:14:03 up 1:35, 1 user, load average: 1.65, 0.98, 0.55
threads: 191 total, 3 running, 98 sleeping, 0 stopped,
Cpu(s): 48.7 us, 0.9 sy, 0.0 ni, 49.4 id, 0.0 wa, 0.0 hi, 0.9 si, 0.0 st
Mem : 5842.6 total, 66.5 free, 4269.4 used, 1781.9 buff/cache
Mem Swap : 2048.0 total, 2047.8 free, 0.2 used, 1633.2 avail Mem

  PID USER      PR  NI    VIRT    RES    SHR     S    %CPU  %MEM     time+   COMMAND
2091 ollama    20   0 3551832  2.8g 13828 R 99.9 49.5  6:04.31 llama-server
2103 ollama    20   0 3551832  2.8g 13820 R 99.7 49.5  5:41.76 llama-server
1973 mariana    20   0 6438812  1.1g 422192 S  0.3 19.8  0:29.23 python
2302 mariana    20   0 8356    6180 3980 R  0.3 0.1  0:00.15 top
1 root        20   0 241660 15384 11564 S  0.0 0.3  0:02.25 systemd
2 root        20   0 3188    2288 2972 S  0.0 0.0  0:00.07 init-systemd(Ub
9 root        20   0 3188    2288 2972 S  0.0 0.0  0:00.00 Interp
7 root        20   0 3188    2044 1940 S  0.0 0.0  0:00.00 init
8 root        20   0 3188    2044 1940 S  0.0 0.0  0:00.00 init
53 root       19  -1 58480 17112 15784 S  0.0 0.3  0:01.04 systemd-journal
88 systemd+   20   0 22416 14516 12628 S  0.0 0.2  0:00.19 systemd-resolve
97 root        20   0 35252 11912 8988 S  0.0 0.2  0:06.09 systemd-udev
117 root       20   0 2688    3088 1884 S  0.0 0.0  0:00.06 chronyd-starte
118 root       20   0 4472    3148 2900 S  0.0 0.1  0:00.05 cron
119 message+   20   0 8524    5392 4692 S  0.0 0.1  0:00.13 dbus-daemon
147 root       20   0 44096 29872 14436 S  0.0 0.5  0:00.32 networkd-dispat
148 ollama     20   0 2832528 32700 19888 S  0.0 0.5  0:00.09 ollama
192 ollama     20   0 2832528 32700 19888 S  0.0 0.5  0:00.18 ollama
193 ollama     20   0 2832528 32700 19888 S  0.0 0.5  0:00.30 ollama
194 ollama     20   0 2832528 32700 19888 S  0.0 0.5  0:00.00 ollama
195 ollama     20   0 2832528 32700 19888 S  0.0 0.5  0:00.00 ollama
199 ollama     20   0 2832528 32700 19888 S  0.0 0.5  0:00.00 ollama
290 ollama     20   0 2832528 32700 19888 S  0.0 0.5  0:00.60 ollama
204 ollama     20   0 2832528 32700 19888 S  0.0 0.5  0:00.62 ollama
265 ollama     20   0 2832528 32700 19888 S  0.0 0.5  0:00.18 ollama
265 ollama     20   0 2832528 32700 19888 S  0.0 0.5  0:00.18 ollama
2105 ollama     20   0 2832528 32700 19888 S  0.0 0.5  0:00.03 ollama
156 root       20   0 18136 9408 5168 S  0.0 0.2  0:00.22 systemd-logind
168 syslog     20   0 220548 6000 4668 S  0.0 0.1  0:00.10 rsyslogd
238 syslog     20   0 220548 6000 4668 S  0.0 0.1  0:00.05 in:imsock
239 syslog     20   0 220548 6000 4668 S  0.0 0.1  0:00.01 in:imlog
240 syslog     20   0 220548 6000 4668 S  0.0 0.1  0:00.03 rs:main Q:Reg
281 chronyd     20   0 20424 6384 5604 S  0.0 0.1  0:00.06 chronyd
286 chronyd     20   0 12996 9440 1792 S  0.0 0.0  0:00.01 chronyd
382 root       20   0 123468 32468 17860 S  0.0 0.5  0:00.28 unattended-upgr
324 root       20   0 123468 32468 17860 S  0.0 0.5  0:00.00 gmain
386 root       20   0 5492    2780 2592 S  0.0 0.0  0:00.02 aptetty
382 root       20   0 8648    5636 4792 S  0.0 0.1  0:00.02 login
432 mariana    20   0 22520 13064 10956 S  0.0 0.2  0:00.25 systemd
434 mariana    20   0 22516 4088 2888 S  0.0 0.1  0:00.00 (sd-pam)
459 mariana    20   0 6136    4972 3432 S  0.0 0.1  0:00.05 bash
965 root       20   0 3106    1116 800 S  0.0 0.0  0:00.00 SerialReader
967 root       20   0 3196    1252 1188 S  0.0 0.0  0:00.00 relay(971)
971 mariana    20   0 6268    5296 3624 S  0.0 0.1  0:00.35 bash
1737 polkitd    20   0 306400 8600 7588 S  0.0 0.1  0:00.09 polkitd
1739 polkitd    20   0 306400 8600 7588 S  0.0 0.1  0:00.07 gmain
1740 polkitd    20   0 306400 8600 7588 S  0.0 0.1  0:00.00 pool-spawner
```

http://localhost:8080/c/0013e4a1...

System Processes ...

Explique em três frases o que é um processo em um sistema operacional.

ollama3.2:3b

Um processo é uma unidade de execução em um sistema operacional, que executa uma tarefa específica. Quando um processo é criado, ele recebe uma parte do espaço de memória do sistema e tem seu próprio contexto de execução. O sistema operacional gerencia os processos, garantindo que eles compartilhem recursos limitados e não sobrecarreguem o sistema.

Explique em poucas frases o que é uma thread em um sistema operacional.

ollama3.2:3b

|

Enviar uma Mensagem

ollama3.2:3b

10.4. Análise

A comparação entre os dois momentos evidencia a diferença entre o Ollama em estado de espera e o Ollama realizando uma inferência. Antes da requisição, o processo principal permanecia ativo, mas apresentava baixa utilização de CPU. Durante a geração da resposta, foi identificado o processo llama-server, associado ao Ollama, com utilização elevada de CPU e múltiplas threads.

Esse comportamento é compatível com a execução da inferência local: o Open WebUI atua como camada de interface, o Ollama gerencia a execução do modelo e o llama-server realiza o processamento necessário para gerar a resposta.

As quantidades de threads, utilização de CPU e consumo de memória são medidas dependentes do instante da coleta e podem variar entre diferentes requisições.

11. COMUNICAÇÃO E PORTAS

11.1. API do Ollama

A comunicação entre a camada de aplicação e o runtime foi analisada considerando a API HTTP local do Ollama.

Porta observada: 11434

Endereço: 127.0.0.1 (localhost)

Comando utilizado: ss -lntp | grep -E '11434|8080'

Resultado relevante:

```
LISTEN 0 4096 127.0.0.1:11434 0.0.0.0:*
```

A saída confirma que o Ollama estava escutando na porta TCP 11434, vinculada ao endereço local 127.0.0.1. Dessa forma, o serviço estava disponível para comunicação local no ambiente WSL2.

Evidência:

```
mariana@Lenovo:~$ ss -lntp | grep -E '11434|8080'
LISTEN 0      2048          0.0.0.0:8080      0.0.0.0:*
users: (("python",pid=1973,fd=32))
LISTEN 0      4096        127.0.0.1:11434  0.0.0.0:*
```

11.2. Análise

A arquitetura utilizada estabelece uma comunicação local entre o Open WebUI e o Ollama. O usuário envia a solicitação por meio do navegador para o Open WebUI, que funciona como camada de interface e aplicação. O Open WebUI encaminha a solicitação ao Runtime Ollama por meio de comunicação HTTP local, utilizando a API disponibilizada na porta 11434.

Nesse processo, a porta TCP funciona como ponto de comunicação entre os componentes. O endereço 127.0.0.1 indica que essa comunicação ocorre localmente no ambiente, sem necessidade de expor o serviço de inferência diretamente à rede externa.

Do ponto de vista de Sistemas Operacionais, essa comunicação envolve processos independentes que utilizam mecanismos de comunicação por rede local, com sockets TCP. O Ollama permanece como processo servidor aguardando conexões e requisições, enquanto o Open WebUI atua como cliente da API durante o encaminhamento das solicitações.

A utilização de uma API HTTP local também permite separar as responsabilidades da aplicação: o Open WebUI fornece a interface para o usuário, enquanto o Ollama gerencia a execução do modelo e os recursos necessários para a inferência.

12. CHAMADAS DE SISTEMA

12.1. Metodologia

Foi utilizado o strace para observar as chamadas de sistema realizadas durante uma execução controlada do Ollama com o modelo llama3.2:3b.

Inicialmente, foi verificada a instalação da ferramenta por meio do comando:

```
strace --version
```

```
mariana@Lenovo:~$ strace --version
strace -- version 6.19
Copyright (c) 1991-2026 The strace developers <https://strace.io>.
This is free software; see the source for copying conditions. There is
NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

Optional features enabled: stack-trace=libunwind stack-demangle m32-mpers
s mx32-mpers
```

Foi utilizada a **versão 6.19** do strace.

Para realizar a coleta, foi executado:

```
strace -f -c -o ~/strace-resumo.txt ollama run llama3.2:3b "Explique em poucas frases o que é uma thread em um sistema operacional."
```

```
mariana@Lenovo:~$ strace -f -c -o ~/strace-resumo.txt ollama run llama3.2:3b "Explique em poucas frases o que é uma thread em um sistema operacional."
Uma thread é uma unidade de execução de código em um sistema operacional, que executa partes do código como se fossem processos independentes, mas compartilhando recursos e memória.
```

A opção -f permitiu acompanhar processos e threads relacionados à execução, enquanto -c realizou a agregação das chamadas de sistema e seus respectivos tempos. O resultado foi armazenado no arquivo ~/strace-resumo.txt.

```
mariana@Lenovo:~$ cat ~/strace-resumo.txt
% time      seconds      usecs/call      calls      errors  syscall
-----
73.06      2.105910          1385          1520          300  futex
11.48      0.330986           303          1089           4  epoll_pwait
10.42      0.300303           194          1546           6  nanosleep
 1.67      0.048073           91           526           6  write
 0.81      0.023374           82           285           1  ioctl
 0.45      0.013050           63           204          47  read
 0.29      0.008243          128           64           4  rt_sigreturn
 0.27      0.007665          119           64           1  tgkill
 0.23      0.006741          105           64           1  getpid
 0.15      0.004188           46           91           1  mmap
 0.13      0.003853           96           40           1  pread64
 0.13      0.003841           60           63           1  rt_sigprocmask
 0.13      0.003710           46           80           1  sched_yield
 0.11      0.003223          358           9           1  clone3
 0.09      0.002503           80           31           1  sched_getaffinity
 0.09      0.002465          2465           1           1  execve
 0.08      0.002442           66           37           1  prctl
 0.06      0.001666           14          114           1  rt_sigaction
 0.04      0.001215           36           33          13  newfstatat
 0.04      0.001141          103           11           1  madvise
 0.04      0.001082           54           20           1  sigaltstack
 0.03      0.000931           51           18           1  mprotect
 0.03      0.000888           27           32           1  pselect6
 0.03      0.000876           97           9           1  munmap
 0.03      0.000753           34           22           4  openat
 0.02      0.000497           49           10           1  gettid
```

A execução foi concluída com sucesso e o modelo produziu uma resposta sobre o conceito de thread.

12.2. Resultados

Chamada / família	Função	Relação com a aplicação	Evidência
futex	Sincronização e espera entre threads	Relacionada à coordenação das múltiplas	1.520 chamadas; 73,06% do tempo

		threads utilizadas durante a execução do modelo	
epoll_pwait	Aguarda eventos de entrada/saída em descritores	Permite que processos ou threads aguardem eventos de comunicação ou I/O sem realizar espera ocupada	1.089 chamadas; 11,48% do tempo
nanosleep	Suspende a execução por um intervalo de tempo	Representa momentos em que processos ou threads permanecem temporariamente em espera	1.546 chamadas; 10,42% do tempo

Também foram observadas chamadas relacionadas à entrada e saída, como read e write, chamadas de gerenciamento de memória, como mmap, e criação de threads/processos, como clone3.

12.3. Análise

A chamada futex foi a que apresentou maior participação no tempo de sistema observado, correspondendo a 73,06% do tempo registrado pelo strace. Essa chamada é utilizada pelo Linux para mecanismos de sincronização entre threads, sendo compatível com o comportamento observado anteriormente no ps e no top -H, que mostraram múltiplas threads durante a execução do Ollama.

A chamada epoll_pwait corresponde a mecanismos de espera por eventos de entrada e saída. Foram registradas 1.089 ocorrências, representando 11,48% do tempo medido. Sua presença é compatível com aplicações que precisam aguardar eventos de comunicação ou operações de I/O sem manter uma thread continuamente consumindo CPU.

A chamada nanosleep apareceu 1.546 vezes, correspondendo a 10,42% do tempo observado. Ela representa situações em que uma thread ou processo solicita ao sistema operacional uma suspensão temporária de sua execução. Isso demonstra que a execução não consiste apenas em processamento contínuo: também existem períodos de espera e sincronização gerenciados pelo sistema operacional.

Também foram registradas chamadas read e write, relacionadas à leitura e escrita de dados, além de mmap, relacionada ao mapeamento de memória. A chamada clone3, observada nove vezes, está relacionada à criação de novas tarefas no Linux e complementa a evidência de utilização de múltiplas unidades de execução.

A coleta apresentou 382 chamadas com retorno de erro. A existência de erros no resumo do strace não significa, por si só, que a execução da aplicação tenha falhado, pois algumas chamadas podem retornar erros esperados durante tentativas de consulta, configuração ou operação de recursos. A execução do modelo foi concluída normalmente e produziu a resposta esperada.

13. METODOLOGIA DOS EXPERIMENTOS

Foram definidas três configurações experimentais utilizando o mesmo modelo (llama3.2:3b) e a mesma camada de aplicação, composta pelo Open WebUI e pelo Ollama. As configurações foram comparadas utilizando entradas curta e longa, com duas repetições por cenário, totalizando 12 execuções mensuráveis.

13.1. Configuração 1 — Execução padrão

Descrição: execução do modelo utilizando a configuração padrão definida para o experimento, sem alteração específica de concorrência ou do tamanho do contexto.

Carga: uma requisição por vez, utilizando entradas curta e longa.

Contexto: 4096 tokens.

Número de execuções: 4.

13.2. Configuração 2 — Concorrência/carga controlada

Descrição: Execução de duas requisições simultâneas para avaliar o comportamento do sistema sob concorrência controlada.

Alteração realizada: aumento da quantidade de requisições executadas simultaneamente, mantendo o mesmo modelo e os mesmos tipos de entrada.

Número de requisições simultâneas: 2.

Número de execuções: 4.

13.3. Configuração 3 — Ajuste de execução

Descrição: execução do mesmo modelo com redução controlada do tamanho do contexto, permitindo observar o impacto desse parâmetro sobre o desempenho e o consumo de recursos.

Parâmetro alterado: num_ctx.

Valor: 2048 tokens.

Número de execuções: 4.

14. TABELA GERAL DOS EXPERIMENTOS

Execu ção	Configu ração	Entrada	Rep.	Tempo total (s)	Load (s)	Tokens de entrada	Tokens de saída	Tokens /s	Erro
1	C1	Curta	1	25,151	14,40 1	53	34	5,23	OK

2	C1	Curta	2	13,861	0,015	53	64	4,78	OK
3	C1	Longa	1	18,942	0,009	117	64	5,14	OK
4	C1	Longa	2	12,741	0,005	117	64	5,14	OK
5	C2	Curta	1	27,138	0,003	53	64	4,60	OK
6	C2	Curta	2	13,041	0,002	53	64	5,04	OK
7	C2	Longa	1	13,191	0,004	117	64	4,97	OK
8	C2	Longa	2	27,371	0,004	117	64	4,59	OK
9	C3	Curta	1	22,423	13,05 6	53	32	5,85	OK
10	C3	Curta	2	11,722	0,003	53	64	5,62	OK
11	C3	Longa	1	20,622	0,002	117	64	4,52	OK
12	C3	Longa	2	13,234	0,054	117	64	4,97	OK

Observação: O TTFT (Time to First Token) não foi mensurado na coleta principal, pois as requisições foram realizadas utilizando a API do Ollama com stream:false, que retorna a resposta de forma não incremental. O tempo total, a quantidade de tokens de entrada e saída e a taxa de tokens por segundo foram obtidos diretamente durante as execuções.

Os valores de CPU, memória e threads não foram repetidos individualmente para cada execução, pois a coleta desses recursos foi realizada como uma amostra pontual durante uma execução controlada. Os valores observados foram apresentados separadamente na seção de resultados.

15. RESULTADOS

15.1. Tempo de execução

As 12 execuções apresentaram tempos totais entre 11,722 s e 27,371 s. A configuração C1 apresentou média de aproximadamente 17,67 s, enquanto C2 apresentou média de 20,19 s e C3 apresentou média de 17,00 s. A configuração C2 apresentou maior variação entre as repetições, comportamento observado durante a execução de duas requisições simultâneas.

Observação: A evidência da coleta de CPU, memória e threads é apresentada no Anexo A.

15.2. Uso de CPU

Durante uma execução controlada do modelo, o processo llama-server apresentou **42,2% de utilização de CPU** no instante da coleta. A coleta foi realizada por meio do comando ps, durante a inferência do modelo.

O processo llama-server apresentou também 11 threads ativas, indicando que a execução do modelo utilizou múltiplas unidades de execução.

O valor representa uma amostra pontual durante a inferência e pode variar conforme a carga e o instante da medição.

15.3. Uso de memória

Durante a execução controlada, o processo llama-server apresentou **42,5% de utilização de memória**, conforme coleta realizada pelo comando ps.

Esse valor corresponde ao percentual de memória do ambiente atribuído ao processo no momento da coleta. Por se tratar de uma medição pontual, o consumo pode variar conforme o estado do modelo, o tamanho da entrada e a carga de trabalho.

15.4. Threads

Durante a inferência, o processo llama-server apresentou **11 threads**, conforme identificado pelo campo NLWP do comando ps.

A observação é consistente com as análises anteriores de processos e threads realizadas no ambiente, nas quais o servidor do modelo também apresentou múltiplas threads durante o processamento.

A quantidade observada representa a situação no momento da coleta e pode variar de acordo com a execução.

15.5. Armazenamento

Item	Tamanho medido
Modelo llama3.2:3b	1,9 GB
Open WebUI	1,6 MB

experimentos_final.csv	4,0 KB
strace-resumo.txt	4,0 KB

15.6. Taxa de erro

As 12 execuções experimentais foram concluídas com sucesso, sem ocorrência de erro registrada no campo correspondente da coleta.

Fórmula utilizada:

$$\text{Taxa de erro} = (\text{número de execuções com erro} / \text{número total de execuções}) \times 100$$

Resultado:

$$\text{Taxa de erro} = (0 / 12) \times 100 = 0\%$$

16. CRITÉRIO DE QUALIDADE DAS RESPOSTAS

Para avaliar a qualidade das respostas produzidas pelo modelo, foi utilizado um critério simples, baseado na relação entre o conteúdo solicitado no prompt e a resposta apresentada pelo modelo:

0 — Incorreta ou inadequada: a resposta não responde à pergunta, apresenta informações incorretas ou permanece fora do tema solicitado.

1 — Parcialmente adequada: a resposta aborda o tema solicitado, mas apresenta omissões, explicações incompletas ou alguma imprecisão relevante.

2 — Adequada: a resposta responde diretamente ao que foi solicitado, apresenta informações corretas e suficientes e mantém coerência com o tema do prompt.

O mesmo critério deve ser aplicado às diferentes configurações experimentais, sem considerar o tempo de execução como fator de qualidade da resposta.

A avaliação de qualidade foi tratada separadamente das métricas de desempenho. Dessa forma, uma resposta mais rápida não é considerada automaticamente melhor do ponto de vista qualitativo.

17. ANÁLISE DOS RESULTADOS

17.1. Relação entre modelo e recursos

O modelo utilizado foi o llama3.2:3b, com aproximadamente 3,2 bilhões de parâmetros, formato GGUF e quantização Q4_K_M. O modelo ocupou aproximadamente 1,9 GB no armazenamento do ambiente WSL2.

Durante uma execução controlada, o processo llama-server apresentou 42,2% de utilização de CPU e 42,5% de utilização de memória, com 11 threads. Esses valores demonstram que a execução da inferência local representou uma carga significativa para os recursos disponíveis no ambiente experimental, que possuía 2 núcleos físicos e 4 threads de CPU e aproximadamente 5,7 GiB de memória disponíveis ao WSL.

A quantização Q4_K_M contribuiu para manter o tamanho do modelo compatível com o ambiente utilizado, permitindo sua execução local sem a necessidade de uma GPU dedicada disponível para o Ollama.

17.2. Processos e threads

Durante a análise dos processos, o Ollama foi identificado como processo principal do serviço, enquanto o processamento do modelo ocorreu por meio do processo llama-server, associado ao Ollama.

Durante a inferência, o llama-server apresentou 11 threads e chegou a apresentar 42,2% de utilização de CPU na coleta representativa. A utilização de múltiplas threads permitiu que o processamento do modelo fosse distribuído entre as unidades de execução disponíveis no processador.

A quantidade de threads observada corresponde à situação registrada no momento da coleta e pode variar de acordo com o estado da aplicação e da carga de trabalho.

17.3. Concorrência

A configuração C2 foi utilizada para avaliar o comportamento do sistema com duas requisições simultâneas. A média do tempo total por requisição foi de aproximadamente 20,19 segundos, enquanto a C1 apresentou média de aproximadamente 17,67 segundos.

Além da média superior, a C2 apresentou maior variação entre algumas execuções. As execuções 5 e 8 apresentaram tempos de 27,138 s e 27,371 s, respectivamente, enquanto outras execuções da mesma configuração apresentaram tempos próximos de 13 segundos.

Os resultados indicam que a execução concorrente aumentou a variabilidade e, em algumas requisições, o tempo individual de resposta. Entretanto, não foi realizada uma medição específica da vazão do lote completo de requisições, portanto não é possível afirmar, com os dados coletados, que a concorrência aumentou ou reduziu a vazão global do sistema.

17.4. Entradas maiores

Foram utilizadas duas entradas com tamanhos diferentes. A entrada curta apresentou 53 tokens, enquanto a entrada longa apresentou 117 tokens, conforme informado pela API do Ollama durante as execuções.

Apesar de a entrada longa possuir mais tokens, o aumento do tamanho do prompt não produziu um aumento uniforme no tempo total. Isso pode ser observado nas diferentes configurações, nas quais os tempos variaram também em função do estado de carregamento do modelo e da carga do sistema.

Assim, os resultados indicam que o tamanho da entrada influencia a quantidade de processamento necessária, mas, neste experimento, não foi o único fator responsável pela variação do tempo de resposta.

Também é importante distinguir o tamanho efetivo do prompt do parâmetro `num_ctx`: os valores de 4096 e 2048 tokens representam o limite de contexto configurado para cada cenário, e não a quantidade efetivamente utilizada pelos prompts, que foi de 53 ou 117 tokens.

17.5. CPU e memória

A coleta representativa durante a inferência registrou 42,2% de CPU e 42,5% de memória para o processo llama-server.

Como o ambiente possuía recursos computacionais limitados, a execução concorrente de duas requisições poderia aumentar a disputa pelas unidades de processamento e pela memória disponíveis. A maior variação observada nos tempos da configuração C2 é compatível com esse comportamento.

Entretanto, como CPU e memória não foram registradas individualmente em cada uma das 12 execuções, esses valores devem ser interpretados como uma amostra representativa e não como médias de todo o experimento.

17.6. Armazenamento

O modelo llama3.2:3b ocupou aproximadamente 1,9 GB no armazenamento do ambiente WSL2. A pasta utilizada pelo Open WebUI apresentou aproximadamente 1,6 MB no momento da medição.

Os arquivos de resultados também apresentaram pequeno consumo de armazenamento: aproximadamente 4,0 KB para `experimentos_final.csv` e 4,0 KB para `strace-resumo.txt`.

Dessa forma, o principal componente de armazenamento entre os itens medidos foi o próprio modelo de linguagem. A utilização do formato GGUF e da quantização Q4_K_M permitiu manter o modelo em um tamanho compatível com o ambiente experimental.

17.7. Escalonamento

O comportamento observado está relacionado ao escalonamento de CPU porque o processamento do modelo utiliza múltiplas threads e compete pelas unidades de execução disponíveis no processador.

Na configuração C2, duas requisições foram executadas simultaneamente. Nesse cenário, as threads relacionadas ao processamento do modelo precisam compartilhar os recursos de CPU disponíveis. A variação dos tempos individuais, especialmente nas execuções de aproximadamente 27 segundos, é compatível com a existência de maior competição por recursos durante a execução concorrente.

O experimento não teve como objetivo medir diretamente o algoritmo específico de escalonamento utilizado pelo sistema operacional. A análise foi baseada nos efeitos observáveis da concorrência sobre os processos, threads e tempos de execução.

18. RESPOSTAS ÀS QUESTÕES DE ANÁLISE

18.1. Por que a trilha escolhida é adequada à disciplina?

A Trilha A — Chat local: Ollama + Open WebUI — é adequada à disciplina porque permite observar, em um ambiente controlado, conceitos relacionados a processos, threads, escalonamento, comunicação entre processos, utilização de CPU e memória e chamadas de sistema. A arquitetura também permite observar a comunicação entre a aplicação Open WebUI e o runtime Ollama por meio de uma API HTTP local.

18.2. Qual modelo foi selecionado, quais são seus parâmetros e por que foi escolhido?

Foi selecionado o modelo llama3.2:3b, da família Llama 3.2, disponibilizado pelo Ollama. O modelo possui aproximadamente 3,2 bilhões de parâmetros, utiliza formato GGUF e quantização Q4_K_M. O contexto máximo informado pelo modelo é de 131.072 tokens.

A escolha foi baseada principalmente na restrição de até 10 bilhões de parâmetros estabelecida pela atividade e nas limitações de hardware do ambiente experimental. A quantização Q4_K_M também contribui para reduzir o tamanho necessário para armazenamento e tornar a execução local mais adequada ao ambiente disponível.

18.3. Como formato, quantização, contexto e tamanho afetaram armazenamento, RAM ou VRAM?

O modelo utiliza o formato GGUF e quantização Q4_K_M, apresentando aproximadamente 1,9 GB de armazenamento no ambiente utilizado. A quantização reduz o tamanho necessário em comparação com representações de maior precisão, favorecendo a execução em um computador com recursos limitados.

O modelo possui contexto máximo de 131.072 tokens, porém os experimentos utilizaram configurações de num_ctx de 4096 e 2048 tokens. Esses valores foram definidos como parâmetros experimentais. As entradas efetivamente utilizadas foram menores, com 53 e 117 tokens.

Não foi identificada uma GPU NVIDIA disponível para utilização pelo Ollama no ambiente WSL2, portanto não foi realizada medição de VRAM dedicada.

18.4. Quais processos e threads foram observados na inicialização e na inferência?

Na inicialização foi identificado o processo principal do Ollama, com PID 148. Durante a inferência foi identificado o processo llama-server, associado ao Ollama, responsável pelo processamento do modelo.

Em uma coleta representativa durante a inferência, o llama-server apresentou 11 threads e utilização de aproximadamente 42,2% da CPU e 42,5% da memória. A análise com ps -eLf, pstree e top -H permitiu observar a existência de múltiplas threads durante o processamento.

18.5. Quais chamadas de sistema foram relevantes para carga, leitura de dados, comunicação ou logs?

A coleta com strace -f -c registrou 6.080 chamadas de sistema durante a execução controlada. Entre as chamadas mais relevantes estavam futex, epoll_pwait, nanosleep, read, write, mmap e clone3.

A chamada futex apresentou grande quantidade de ocorrências e está relacionada a mecanismos de sincronização entre threads. epoll_pwait está relacionada à espera por eventos, enquanto nanosleep representa períodos de espera. As chamadas read e write estão relacionadas a operações de leitura e escrita, e mmap ao mapeamento de regiões de memória. clone3 está associado à criação de novos contextos de execução.

A presença dessas chamadas é compatível com uma aplicação multithread que realiza processamento, sincronização, espera por eventos e operações de memória e entrada/saída.

18.6. O aumento de concorrência melhorou responsividade ou vazão? Quais custos surgiram?

Nos dados coletados, a configuração C2, com duas requisições simultâneas, apresentou média de aproximadamente 20,19 segundos por requisição, enquanto o C1 apresentou aproximadamente 17,67 segundos.

Também foram observadas execuções de 27,138 e 27,371 segundos na C2. Isso indica que a concorrência aumentou a variabilidade e, em algumas requisições, aumentou o tempo individual de resposta.

Não é possível afirmar que houve aumento de vazão, pois o tempo total do lote de duas requisições não foi medido como uma métrica específica. O principal custo observado foi a maior variação e o aumento do tempo de algumas requisições individuais.

18.7. Houve competição por CPU, memória, armazenamento, GPU ou rede?

Foi observada utilização significativa de CPU e memória pelo llama-server, com uma coleta representativa de 42,2% de CPU e 42,5% de memória.

A configuração C2 introduziu concorrência entre duas requisições e apresentou maior variação nos tempos individuais, sendo compatível com maior disputa pelos recursos computacionais disponíveis.

Não foi identificada GPU NVIDIA disponível ao ambiente experimental e, portanto, não foi possível avaliar a competição por VRAM. A comunicação entre Open WebUI e Ollama ocorreu localmente, pela interface localhost, não sendo observada dependência de uma comunicação externa para a inferência.

18.8. Como entradas ou contextos maiores afetaram desempenho e recursos?

A entrada curta possuía 53 tokens e a entrada longa 117 tokens. Apesar de a entrada longa possuir mais tokens, os resultados não apresentaram aumento uniforme do tempo de execução em todas as configurações.

Isso demonstra que o tempo de resposta também foi influenciado por outros fatores, como carregamento inicial do modelo, concorrência e estado dos recursos do ambiente.

Além disso, os valores de num_ctx utilizados nas configurações C1 e C3 foram 4096 e 2048, respectivamente. Esses valores representam o limite de contexto configurado, e não o tamanho efetivo dos prompts utilizados.

18.9. Como a arquitetura local contribui para privacidade, disponibilidade e controle de dados?

A arquitetura utilizada mantém o processamento do modelo no ambiente local, com o Open WebUI e o Ollama executando no WSL2 e a comunicação principal ocorrendo por interfaces locais.

Essa arquitetura reduz a necessidade de enviar os prompts para uma API externa durante a inferência, proporcionando maior controle sobre os dados processados. Também permite utilizar o sistema mesmo sem depender de um serviço externo de inferência, desde que a aplicação, o modelo e os recursos necessários estejam disponíveis localmente.

A privacidade efetiva ainda depende das demais configurações e componentes utilizados no ambiente, mas a execução local oferece maior controle sobre o fluxo dos dados.

18.10. Quais limitações ameaçam a validade dos resultados?

Entre as principais limitações estão a quantidade reduzida de repetições, os recursos limitados de CPU e memória, a utilização de WSL2 em vez de uma instalação Linux nativa e a ausência de uma GPU NVIDIA disponível para comparação.

Também foram realizadas medições pontuais de CPU, memória e threads, e não uma coleta contínua durante todas as 12 execuções. O TTFT não foi mensurado na coleta principal porque foi utilizado `stream:false`.

Além disso, a configuração C2 avaliou duas requisições simultâneas, mas não foi realizada uma medição específica de vazão do conjunto de requisições. Outros processos do ambiente também podem ter introduzido pequenas variações nos resultados.

18.11. Quais informações geradas por IA precisaram ser verificadas, corrigidas ou rejeitadas?

As ferramentas de IA foram utilizadas como apoio à compreensão dos requisitos da atividade, à organização do experimento, à elaboração de comandos e à estruturação do relatório. As informações técnicas foram verificadas por meio da execução dos comandos no ambiente experimental e pela comparação com os resultados efetivamente observados.

Um exemplo de correção ocorreu na definição dos parâmetros `num_ctx` de 4096 e 2048 tokens. Esses valores foram tratados como escolhas experimentais, e não como valores medidos ou como características obrigatórias do modelo. Os valores efetivamente observados de 53 e 117 tokens para as entradas foram obtidos diretamente pela API do Ollama.

Também foram evitadas conclusões que não poderiam ser comprovadas pelos dados coletados, como afirmar aumento de vazão na configuração concorrente sem uma medição específica de throughput.

19. USO CRÍTICO DE IA GENERATIVA

19.1. Ferramentas utilizadas

Ferramenta/modelo	Finalidade
ChatGPT - GPT-5.6 Luna	Apoio à interpretação dos requisitos da atividade, planejamento dos experimentos, elaboração e revisão de comandos, análise dos resultados e organização do relatório

Llama 3.2 3B - llama3.2:3b Modelo de linguagem utilizado no experimento de inferência local

19.2. Prompts relevantes

Prompt 1

“Explique em poucas frases o que é um processo em um sistema operacional e qual é a diferença entre processo e thread.”

Esse prompt foi utilizado na primeira execução do modelo para verificar seu funcionamento e observar a resposta inicial.

Prompt 2

“Explique em poucas frases o que é um processo e qual a diferença entre processo e thread em um sistema operacional.”

Esse prompt foi utilizado como entrada curta nas configurações C1, C2 e C3, com duas repetições em cada configuração.

Prompt 3

“Explique os conceitos de processo, thread, escalonamento de CPU e concorrência em sistemas operacionais. Descreva como processos e threads utilizam recursos computacionais, como CPU e memória, e explique de forma objetiva como o sistema operacional coordena a execução de múltiplas threads. Apresente também um exemplo simples de situação em que dois processos precisam compartilhar recursos.”

Esse prompt foi utilizado como entrada longa nas configurações C1, C2 e C3, também com duas repetições em cada configuração.

19.3. Sugestões aproveitadas

Foram aproveitadas sugestões relacionadas à organização do ambiente experimental, estruturação das três configurações, elaboração do script de coleta, organização das evidências, estrutura do relatório e interpretação dos resultados.

Também foram aproveitadas orientações para utilizar comandos como ps, ps -eLf, pstree, strace e consultas à API local do Ollama.

19.4. Sugestões corrigidas

As sugestões foram corrigidas quando não correspondiam diretamente aos dados medidos no ambiente. Um exemplo foi a distinção entre os valores de num_ctx utilizados nos experimentos e a quantidade efetiva de tokens dos prompts.

Os valores de 4096 e 2048 tokens foram utilizados como parâmetros experimentais, enquanto os tamanhos efetivos das entradas, de 53 e 117 tokens, foram obtidos pela própria API durante as execuções.

Também foi corrigida a forma de apresentação das métricas de CPU, memória e threads, pois essas informações foram obtidas em uma coleta pontual e não individualmente em cada uma das 12 execuções.

19.5. Sugestões rejeitadas

Foram rejeitadas ou não utilizadas conclusões que não poderiam ser sustentadas diretamente pelas evidências coletadas. Entre elas está a atribuição de valores de TTFT às 12 execuções sem uma medição específica e a conclusão de aumento de vazão na configuração concorrente sem uma medição de throughput.

Também não foram utilizados dados de GPU que não pudessem ser comprovados pelo ambiente experimental.

19.6. Verificação

As informações obtidas por meio de ferramentas de inteligência artificial foram confrontadas com documentação, comandos executados no ambiente experimental, resultados observados e demais evidências coletadas durante a atividade.

Os dados quantitativos apresentados no relatório, como tempos de execução, quantidade de tokens, taxa de tokens por segundo, consumo de recursos, quantidade de threads, chamadas de sistema e armazenamento, foram baseados nas medições realizadas no ambiente experimental.

20. LIMITAÇÕES

Entre as limitações do experimento estão:

- recursos limitados de CPU e memória disponíveis no ambiente WSL2;
- utilização de WSL2 em vez de uma instalação Linux nativa;
- ausência de uma GPU NVIDIA disponível para aceleração no ambiente experimental;
- impossibilidade de comparar o desempenho da inferência em CPU com uma GPU dedicada;
- variações decorrentes de outros processos executados no sistema durante as medições;
- quantidade limitada de repetições por cenário;
- possíveis variações de temperatura, frequência e carga da CPU;
- características específicas da versão do Ollama e do modelo llama3.2:3b utilizados;
- TTFT não mensurado na coleta principal, devido à utilização da API com `stream:false`;
- ausência de uma medição específica de throughput para o conjunto de requisições concorrentes;
- CPU, memória e quantidade de threads analisadas por meio de uma coleta representativa, e não como médias registradas individualmente nas 12 execuções;
- ausência de comparação com outros modelos ou outras arquiteturas de hardware.

Essas limitações devem ser consideradas na interpretação dos resultados. Dessa forma, os resultados obtidos representam o comportamento observado no ambiente experimental utilizado e não devem ser generalizados diretamente para computadores com configurações de hardware, versões de software ou condições de execução diferentes.

21. REPRODUTIBILIDADE

Para reproduzir o experimento, é necessário possuir um ambiente compatível com os seguintes componentes:

- **Windows com WSL2 habilitado;**
- **Ubuntu 26.04.1 LTS;**
- **Ollama 0.34.2;**
- **Open WebUI v0.11.4;**
- **Python 3.11**, utilizado na execução do Open WebUI;
- **uv**, utilizado para instalação e execução do Open WebUI;
- **Modelo Meta Llama 3.2 3B Instruct**, executado por meio do Ollama;
- Recursos computacionais compatíveis com a execução local do modelo.

Comandos principais

Verificação do ambiente:

```
uname -a  
cat /etc/os-release  
lscpu  
free -h  
df -h
```

Verificação do Ollama:

```
ollama --version  
ollama pull llama3.2:3b  
ollama list  
ollama run llama3.2:3b
```

Verificação da API do Ollama:

```
curl http://localhost:11434/api/tags
```

Instalação e execução do Open WebUI:

```
curl -Lsf https://astral.sh/uv/install.sh | sh  
source $HOME/.local/bin/env  
uv --version  
DATA_DIR=~/.open-webui uvx --python 3.11 open-webui@latest serve
```

Verificação do servidor Open WebUI:

```
curl -I http://localhost:8080
```

Reprodução dos experimentos

Os experimentos de processos, threads, consumo de CPU e memória, comunicação e chamadas de sistema devem ser executados após a inicialização do Ollama e do Open WebUI, seguindo a metodologia apresentada neste relatório.

Os comandos específicos de monitoramento e coleta de dados serão registrados na seção **Metodologia dos Experimentos**, juntamente com os parâmetros utilizados em cada configuração.

Os scripts utilizados, quando aplicável, serão disponibilizados no repositório GitHub do projeto.

Os pesos do modelo de linguagem não são armazenados no repositório devido ao seu tamanho e às condições de distribuição da licença do modelo.

22. REPOSITÓRIO

O código, scripts, resultados experimentais, evidências e arquivos auxiliares utilizados nesta atividade estão disponíveis no repositório público:

Nome: SO_UFS_2026_2_Silva_Mariana

URL: https://github.com/mariana-js/SO_UFS_2026_2_Silva_Mariana

O repositório contém:

- **README.md:** instruções gerais e descrição do experimento;
- **scripts/:** scripts utilizados para execução das medições;
- **resultados/:** arquivos CSV com os resultados das execuções e resumo das chamadas de sistema;
- **evidencias/:** capturas de tela e demais evidências da execução;
- **configuracoes/:** arquivos relacionados à configuração do ambiente;
- **relatorio/:** versão final do relatório em PDF;
- **VIDEO.md:** identificação e link do vídeo da atividade.

Os arquivos disponibilizados têm como objetivo permitir a inspeção dos procedimentos realizados e facilitar a reprodução do experimento. Não foram incluídos pesos de modelos, credenciais, chaves de API, senhas ou outros dados sensíveis.

23. VÍDEO DA ATIVIDADE

URL: https://youtu.be/SH9_ovTPqbU

Data de gravação: 23/09/2026

O vídeo apresenta:

- identificação do discente;
 - trilha e modelo;
 - execução local;
 - ambiente utilizado;
 - recursos computacionais;
 - arquitetura;
 - processos e threads;
 - chamadas de sistema/monitoramento;
 - resultados;
 - gráfico principal;
 - limitações;
 - decisões adotadas.
-

24. CONCLUSÃO

A atividade permitiu analisar, na prática, a execução local de uma aplicação de inteligência artificial generativa sob a perspectiva de Sistemas Operacionais. A utilização do Open WebUI em conjunto com o Ollama possibilitou observar a interação entre diferentes componentes da aplicação, a comunicação por meio de uma API HTTP local, a criação e utilização de processos e threads e o consumo dos recursos computacionais disponíveis no ambiente WSL2.

A execução do modelo llama3.2:3b, com aproximadamente 3,2 bilhões de parâmetros, formato GGUF e quantização Q4_K_M, demonstrou ser viável no ambiente experimental composto por um processador Intel Core i3-10110U, com 2 núcleos físicos e 4 threads, e aproximadamente 5,7 GiB de memória disponível ao WSL2. O modelo ocupou aproximadamente 1,9 GB de armazenamento. Durante uma coleta representativa de inferência, o processo llama-server apresentou 42,2% de utilização de CPU, 42,5% de memória e 11 threads, evidenciando a utilização significativa dos recursos disponíveis durante o processamento.

Os experimentos realizados nas três configurações totalizaram 12 execuções, todas concluídas sem erro, resultando em uma taxa de erro de 0%. A configuração C1 apresentou tempo médio de aproximadamente 17,67 segundos, a C2, com duas requisições simultâneas, apresentou média de 20,19 segundos, e a C3 apresentou média de 17,00 segundos. A configuração concorrente apresentou maior variação entre as execuções, incluindo tempos de aproximadamente 27 segundos, indicando que a concorrência pode aumentar a variabilidade do tempo de resposta em um ambiente com recursos limitados. Entretanto, como não foi realizada uma medição específica de vazão do conjunto de requisições, não é possível concluir que a concorrência tenha aumentado ou reduzido o throughput do sistema.

A análise das chamadas de sistema também evidenciou mecanismos fundamentais para o funcionamento da aplicação. Foram registradas 6.080 chamadas durante a execução monitorada com strace, destacando-se futex, epoll_pwait e nanosleep, além de chamadas relacionadas a leitura, escrita, memória e criação de tarefas. Esses resultados reforçam a relação entre a execução de uma aplicação de IA e conceitos de Sistemas Operacionais, como sincronização entre threads, espera por eventos, gerenciamento de memória, entrada e saída e escalonamento de tarefas.

Dessa forma, os resultados demonstram que a execução local de modelos de linguagem não depende apenas do modelo em si, mas também da forma como processos, threads, memória, CPU, armazenamento e mecanismos de comunicação são gerenciados pelo sistema operacional. A atividade também evidenciou a importância de realizar medições e interpretar os resultados com cautela, distinguindo dados efetivamente coletados de estimativas ou conclusões que não podem ser comprovadas pelo experimento.

Por fim, apesar das limitações relacionadas ao hardware disponível, ao uso do WSL2, à ausência de GPU NVIDIA, ao número de repetições e à não mensuração do TTFT e do throughput global das requisições concorrentes, o experimento atingiu seu objetivo de relacionar os conceitos estudados na disciplina com uma aplicação real de inferência local. A utilização conjunta de evidências experimentais, monitoramento do sistema e análise crítica das informações geradas por ferramentas de inteligência artificial contribuiu para uma compreensão mais concreta do funcionamento de sistemas de IA executados localmente.

25. REFERÊNCIAS

SILBERSCHATZ, Abraham; GALVIN, Peter B.; GAGNE, Greg. Operating System Concepts. 10. ed. Hoboken: Wiley, 2018.

TANENBAUM, Andrew S.; BOS, Herbert. Modern Operating Systems. 5. ed. Boston: Pearson, 2023.

KERRISK, Michael. The Linux Programming Interface. San Francisco: No Starch Press, 2010.

LINUX MAN-PAGES PROJECT. Linux man-pages. Disponível em: <https://man7.org/linux/man-pages/>. Acesso em: 22 set. 2026.

THE LINUX KERNEL DOCUMENTATION. Scheduler documentation. Disponível em: <https://docs.kernel.org/scheduler/>. Acesso em: 22 set. 2026.

OLLAMA. Ollama. GitHub. Disponível em: <https://github.com/ollama/ollama>. Acesso em: 22 set. 2026.

OLLAMA. Ollama API documentation. Disponível em: <https://docs.ollama.com/api>. Acesso em: 22 set. 2026.

OPEN WEBUI. Open WebUI. GitHub. Disponível em: <https://github.com/open-webui/open-webui>. Acesso em: 22 set. 2026.

OPEN WEBUI. Open WebUI Documentation. Disponível em: <https://docs.openwebui.com/>. Acesso em: 22 set. 2026.

META AI. Llama 3.2. Disponível em: https://www.llama.com/llama3_2/. Acesso em: 22 set. 2026.

META. Llama 3.2 3B Instruct. Hugging Face. Disponível em: <https://huggingface.co/meta-llama/Llama-3.2-3B-Instruct>. Acesso em: 22 set. 2026.

HUGGING FACE. Hugging Face Hub. Disponível em: <https://huggingface.co/>. Acesso em: 22 set. 2026.

NIST. Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile. NIST AI 600-1, 2024. Disponível em: <https://doi.org/10.6028/NIST.AI.600-1>. Acesso em: 22 set. 2026.

26. ANEXOS

Anexo A — Processos e threads

Sucesso ao executar os 12 experimentos para as 3 configurações:

```
mariana@Lenovo:~$ ~/experimentos.sh
=== C1 ===
=== C2 ===
=== C3 ===

=== EXPERIMENTOS CONCLUÍDOS ===
execucao,configuracao,entrada,repeticao,tempo_total_s,load_s,prompt_tokens,output_tokens,tokens_s,erro
1,C1,Curta,1,25.151,14.401,53,34,5.23,OK
2,C1,Curta,2,13.861,0.015,53,64,4.78,OK
3,C1,Longa,1,18.942,0.009,117,64,5.14,OK
4,C1,Longa,2,12.741,0.005,117,64,5.14,OK
6,C2,Curta,2,13.041,0.002,53,64,5.04,OK
5,C2,Curta,1,27.138,0.003,53,64,4.60,OK
7,C2,Longa,1,13.191,0.004,117,64,4.97,OK
8,C2,Longa,2,27.371,0.004,117,64,4.59,OK
9,C3,Curta,1,22.423,13.056,53,32,5.85,OK
10,C3,Curta,2,11.722,0.003,53,64,5.62,OK
11,C3,Longa,1,20.622,0.002,117,64,4.52,OK
12,C3,Longa,2,13.234,0.054,117,64,4.97,OK
```

Evidência da coleta de CPU, memória e threads:

```
mariana@Lenovo: ~  
Every 1.0s: ps -eo pid,ppid,stat,pc...  Lenovo: Tue Sep 22 21:16:08 2026  


| PID  | PPID | STAT | %CPU | %MEM | NLWP | COMMAND         |
|------|------|------|------|------|------|-----------------|
| 4456 | 4455 | R+   | 100  | 0.0  | 1    | ps              |
| 4457 | 4455 | S+   | 100  | 0.1  | 1    | head            |
| 3748 | 148  | Sl   | 50.9 | 42.5 | 11   | llama-server    |
| 4391 | 1    | Ssl  | 1.4  | 0.2  | 8    | wsl-pro-service |
| 4258 | 3587 | S+   | 0.1  | 0.0  | 1    | watch           |
| 3587 | 3581 | Ss   | 0.1  | 0.0  | 1    | bash            |
| 97   | 1    | Ss   | 0.0  | 0.1  | 1    | systemd-udevd   |
| 3705 | 3701 | Ss+  | 0.0  | 0.0  | 1    | bash            |
| 148  | 1    | Ssl  | 0.0  | 0.5  | 11   | ollama          |
| 3581 | 3579 | S    | 0.0  | 0.0  | 1    | Relay(3587)     |
| 53   | 1    | S<s  | 0.0  | 0.2  | 1    | systemd-journal |
| 1    | 0    | Ss   | 0.0  | 0.2  | 1    | systemd         |

  
{"model":"llama3.2:3b","created_at":"2026-09-23T00:15:40.510029959Z","response":"**0 que é uma Thread em um Sistema Operacional?**\n\nEm um sistema operacional, uma thread é uma unidade de execução que executa uma parte do código de um programa. Ela é uma estrutura de execução mais leve do que um processo, pois é uma parte do mesmo processo e compart","done":true,"done_reason":"length","context":[128006,9125,128007,271,38766,1303,33025,2696,25,6790,220,2366,18,271,128009,128006,882,128007,271,45053,2428,297,1744,4046,10832,4617,991,4543,29928,2040,22326,13,128009,128006,78191,128007,271,334,46,1744,4046,10832,8926,991,4543,77354,7613,22326,30,57277,2321,4543,29928,2040,22326,11,10832,4617,4046,10832,653,9252,409,3969,62067,1744,3969,16382,10832,20108,656,45159,409,4543,36860,13,4072,64,4046,10832,30607,79807,409,3969,62067,10071,56589,656,1744,4543,59996,11,58883,4046,10832,20108,656,41349,59996,384,39667],"total_duration":8496194504,"load_duration":2824100,"prompt_eval_count":38,"prompt_eval_cached_count":37,"prompt_eval_duration":149171000,"eval_count":64,"eval_duration":8324833000}mariana@Lenovo: ~$
```